SANDO: Safe Autonomous Trajectory Planning for Dynamic Unknown Environments

Kota Kondo[1], Jesús Tordesillas[2], Jonathan P. How[1]

arXiv:2604.07599v2 [cs.RO] 24 Apr 2026

***Abstract*— This paper presents SANDO, a safe trajectory planner for 3D dynamic unknown environments. In such environments, the locations and motions of obstacles are not known in advance, and a collision-free plan can become unsafe at any moment as obstacles move unpredictably, requiring fast replanning. Existing soft-constraint planners are fast but do not guarantee collision-free paths, while hard-constraint methods typically ensure safety at the cost of longer computation times. SANDO addresses this trade-off through three main contributions. First, to avoid overly conservative or infeasible corridors near dynamic obstacles, a heat map-based A* global planner steers the path away from high-risk regions using soft costs, and a spatiotemporal safe flight corridor (STSFC) generator produces time-layered polytopes that inflate obstacles only by their worst-case reachable set at each time layer, rather than by the worst case over the entire horizon. Second, trajectory optimization is formulated as a Mixed-Integer Quadratic Program (MIQP) with hard collision-avoidance constraints, and a variable elimination technique reduces the number of decision variables, enabling fast computation. Third, a formal safety analysis establishes collision-free guarantees under explicit velocity-bound and estimation-error assumptions. Ablation studies confirm that variable elimination provides up to 7.4× speedup in optimization time, and that STSFCs are critical for maintaining feasibility in dense dynamic environments. Benchmark simulations against state-of-the-art methods across standardized static benchmarks, obstacle-rich forests, and dynamic environments show that SANDO consistently achieves the highest success rate with no constraint violations across all difficulty levels, and additional perception-only experiments without ground truth obstacle information confirm robust performance under realistic sensing conditions. Hardware experiments on a UAV with fully onboard planning, perception, and localization demonstrate six safe flights in static environments and ten safe flights among dynamic obstacles.**

Supplementary Material

Video: https://youtu.be/_T10DJiLQXg
Code: https://github.com/mit-acl/sando.git

I. Introduction

Autonomous unmanned aerial vehicle (UAV) navigation in dynamic unknown environments is challenging: the future trajectories of obstacles are unknown, so a collision-free path can become unsafe moments after it is computed. This requires planners that can quickly recompute safe trajectories while handling spatiotemporal collision avoidance. Existing approaches address parts of this challenge but fall into three categories with distinct limitations (see Table I).

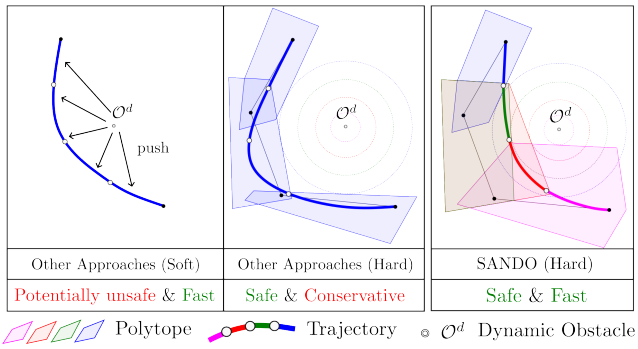


Fig. 1. Comparison of trajectory planning approaches near a dynamic obstacle $\mathcal{O}^d$. **Left (Other Approaches – Soft):** Soft-constraint methods use penalty-based collision avoidance, producing fast but potentially unsafe trajectories. **Center (Other Approaches – Hard):** Other hard-constraint methods inflate the obstacle by its full worst-case reachable set and generate purely spatial corridors, producing safe but overly conservative trajectories that detour far from the obstacle. **Right (SANDO):** SANDO generates spatiotemporal safe flight corridors (STSFCs) that account for when the trajectory passes each region, inflating the obstacle only by its reachable set at the corresponding time layer, yielding a safe and fast trajectory. (The dynamic obstacle is depicted as a point mass for visual clarity; in practice, obstacles have finite axis-aligned bounding box (AABB) extents, and the reachable set is the Minkowski sum of the AABB with a cube whose half-side equals the per-layer reachable radius.)

First, planners designed for unknown static environments (e.g., EGO-Planner [1], FASTER [2], HDSM [3], and SUPER [4]) achieve fast replanning but assume that once space is observed as free, it remains free, making them unsuitable for environments with moving obstacles. Second, planners that handle dynamic obstacles with soft constraints (e.g., HOPA [5], ViGO [6], FAPP [7], Risk-Aware [8], and FHD [9]) can react to moving obstacles but provide no formal safety guarantee, as collision avoidance is encouraged via penalty terms rather than enforced (see Fig. 1). Third, planners that enforce hard safety constraints in dynamic environments (e.g., CC-MPC [10], OA-MPC [11], Liu et al. [12], Stamouli et al. [13], STS [14], and IP-MPC [15]) guarantee safety only at discretized model predictive control (MPC) or sample points, leaving the trajectory potentially unsafe between time steps. While PANTHER [16] provides continuous-time safety in dynamic environments, it has only been demonstrated in open settings and the computation scales poorly with the number of obstacles. As summarized in Table I, no existing method simultaneously (1) handles spatiotemporal dynamic obstacle collision avoidance, (2) guarantees continuous-time safety, and (3) operates in dynamic unknown confined environments. To address these

[1]K. Kondo and J. P. How are with the Departments of Aeronautics and Astronautics, and Mechanical Engineering, Massachusetts Institute of Technology. {kkondo, jhow}@mit.edu.
[2]J. Tordesillas is an Assistant Professor with the Department of Electronics, Automation, and Communications, Comillas ICAI. jtordesillas@comillas.edu.
This work is supported by DSTA.

gaps, we introduce SANDO (**S**afe **A**uto**N**omous trajectory planning for **D**ynamic unkn**O**wn environments), a trajectory planner that integrates a heat map-based A* global planner, a novel Spatiotemporal Safe Flight Corridor (STSFC) generation method, and a variable elimination-based hard-constrained optimization technique to provide continuous-time safety guarantees in dynamic unknown environments.

II. Related Work

A. Environment Taxonomy

We focus on unknown environments, where no prior map is available, and classify them into two categories: (1) static vs. dynamic, and (2) open vs. confined. Static environments have fixed obstacles, whereas dynamic environments include moving obstacles. Open environments have relatively sparse obstacle distributions, whereas confined environments involve occlusions and narrow passages.

We organize our review around the three criteria in Table I, namely, environment type (static vs. dynamic), environment structure (open vs. confined), and safety guarantee level (none, discretized, or continuous). We first discuss planners designed for static environments, and then review dynamic-environment planners grouped by their safety guarantee.

B. Planning in Unknown Static Environments

Several planners are designed for unknown static environments and achieve fast replanning by assuming that once space is observed as free, it remains free. EGO-Planner [1] uses a gradient-based local optimizer with soft collision penalties, enabling fast replanning in both open and confined static settings but providing no safety guarantees. FASTER [2] builds on the MIQP trajectory planning approaches [19], [20] by generating safe flight corridors via convex decomposition and solving a hard-constrained MIQP within them, providing continuous-time collision-free guarantees in static environments; however, it does not model dynamic obstacles. HDSM [3] extends the corridor-based approach with a high-degree spline representation that improves trajectory smoothness and corridor utilization, also providing continuous safety guarantees but only under the static assumption. SUPER [4] adopts FASTER's exploratory-safe trajectory framework and replaces the hard-constraint optimizer with a gradient-based solver for faster computation, but trades off safety guarantees by using soft constraints and is limited to static environments. While these methods work well in static settings, none of them handle dynamic obstacles.

C. Soft-Constrained Planning in Dynamic Environments

A number of planners handle dynamic obstacles but use soft constraints, meaning collision avoidance is encouraged via penalty terms but not strictly enforced. HOPA [5] uses obstacle positions and potential fields to perform avoidance maneuvers, but operates only in open environments and provides no hard safety guarantee. ViGO [6] fuses vision-based obstacle detection with a gradient-based planner to navigate dynamic clutter in both open and confined settings, but relies on soft collision penalties. FAPP [7] tightly integrates perception and planning for dynamic cluttered environments, demonstrating fast replanning and good practical performance; however, its collision avoidance is soft-constrained using MINCO [21], so it does not guarantee safety. Xu et al. [18] propose a heuristic-based incremental probabilistic roadmap (PRM) for efficient replanning in dynamic environments, using ViGO [6] as the underlying local planner and thus inheriting its soft-constraint limitation. Risk-Aware [8] incorporates risk metrics into the planning objective to improve safety awareness in dynamic environments, but the risk terms act as soft penalties rather than hard constraints. FHD [9] uses a learning-based approach to navigate dynamic environments, achieving agile flight without explicit obstacle modeling; however, the learned policy provides no formal safety guarantee. While these methods demonstrate strong empirical performance, the absence of hard constraints means safety cannot be formally guaranteed, which is a critical limitation for safety-critical UAV deployments.

D. Safety-Guaranteed Planning in Dynamic Environments

Early approaches to dynamic collision avoidance include velocity obstacles [22]. Formal safety frameworks such as control barrier functions [23] and Hamilton-Jacobi reachability [24] provide rigorous guarantees but can be computationally expensive in 3D cluttered environments. In the trajectory planning literature, a smaller set of planners enforce collision avoidance as hard constraints in dynamic environments. We distinguish between discretized guarantees (safety enforced only at sampled time steps or MPC knot points) and continuous guarantees (safety enforced continuously along the trajectory).

1) Discretized Safety Guarantees: CC-MPC [10] formulates chance constraints within an MPC framework to handle obstacle uncertainty, providing probabilistic safety guarantees at discretized MPC steps but not between them. OA-MPC [11] provides hard-constraint guarantees at MPC steps via worst-case reachability analysis and accounts for occluded regions, but ignores the temporal motion history of obstacles, leading to excessive conservatism (see Fig. 1). Liu et al. [12] compute tight collision probability bounds and enforce chance constraints at MPC steps, but safety is only guaranteed at discrete time points. Stamouli et al. [13] combine conformal prediction with shrinking-horizon MPC to provide probabilistic safety guarantees at MPC steps without distributional assumptions; however, their approach has only been demonstrated in open environments. STS [14] uses state-time space to handle dynamic environments; however, it is only demonstrated in 2D scenarios and only provides safety guarantees at discretized sample points but not continuously. IP-MPC [15] incorporates intent prediction of dynamic obstacles into an MPC framework with hard constraints at MPC steps, improving avoidance quality but without continuous-time guarantees.

All of these methods share a fundamental limitation: collisions can occur between the discrete time points at

TABLE I. State-of-the-art UAV trajectory planners in unknown environments. Safety guarantee: ✓ = continuous, △ = discretized, ✗ = none.

Method	Environments		Safety Guarantee	Note
	Static/Dynamic	Open/Confined		
CC-MPC [10] (2020)	Both	Both	△	Chance hard constraint at discretized MPC steps
EGO-Planner [1] (2021)	Static	Both	✗	Soft constraint
HOPA [5] (2021)	Both	Open	✗	Soft constraint
FASTER [2] (2022)	Static	Both	✓	Continuous guarantee; Assume static env.
PANTHER [16] (2022)	Both	Open	✓	Continuous guarantee; Probabilistic inflation for obstacle uncertainty
EGO-Swarm2 [17] (2022)	Both	Both	✗	Soft constraint
ViGO [6] (2022)	Both	Both	✗	Soft constraint
OA-MPC [11] (2022)	Both	Both	△	Hard constraint at discretized MPC steps & Temporally conservative (See Fig. 1)
Liu et al. [12] (2023)	Both	Both	△	Chance hard constraint at discretized MPC steps
HDSM [3] (2024)	Static	Both	✓	Continuous guarantee; Assume static env.
FAPP [7] (2024)	Both	Both	✗	Soft constraint
Stamouli et al. [13] (2024)	Both	Open	△	Probabilistic safety guarantee at discretized MPC steps
Xu et al. [18] (2024)	Both	Both	✗	Use ViGO [6] as planner
SUPER [4] (2025)	Static	Both	✗	Soft constraint
FHD [9] (2025)	Both	Both	✗	Learning-based approach
STS [14] (2025)	Both	Both	△	Safety only guaranteed at discretized sample points
IP-MPC [15] (2025)	Both	Both	△	Hard constraint at discretized MPC steps
Risk-Aware [8] (2025)	Both	Both	✗	Soft constraint
SANDO (proposed)	Both	Both	✓	Continuous guarantee; Bound dynamic obstacles' maximum speed

which constraints are enforced, particularly for fast-moving obstacles or long MPC intervals.

2) Continuous Safety Guarantees: PANTHER [16] constructs per-obstacle convex hulls to enforce continuous-time collision avoidance in dynamic unknown environments using probabilistic reachable-set inflation for obstacle uncertainty. However, its per-obstacle convex hull representation becomes computationally expensive as the number of obstacles grows, and it has been demonstrated primarily in open environments without many static obstacles.

E. Contributions

To address the gaps identified above, SANDO makes the following contributions:

1) **STSFC Generation:** A novel Spatiotemporal Safe Flight Corridor generation method that produces time-layered polytope sequences accounting for worst-case obstacle reachable sets at each time layer, addressing the lack of time-varying corridor generation framework for dynamic environments (Section IV).
2) **Heat Map-based Global Planner:** A heat map-based A* planner that assigns soft costs around both static and dynamic obstacles, guiding the global path toward regions where larger STSFC corridors can be generated (Section V).
3) **MIQP Trajectory Optimization with STSFCs:** A hard-constraint MIQP formulation that assigns each trajectory piece to a time-layered STSFC polytope, whose continuous-time collision-free guarantee is established by the formal safety analysis below (Section VI).
4) **Formal Safety Analysis:** Safety guarantees through worst-case reachable set inflation with explicit assumptions, and a discussion of recursive feasibility limitations.
5) **Extensive Evaluation** in simulation across diverse environments and hardware experiments on a fully automated UAV platform, demonstrating the practical effectiveness of SANDO in real-world scenarios.

III. System Overview

SANDO consists of five modules (see Fig. 2): a dynamic obstacle tracker, a heat map generator, a global path planner, an STSFC generator, and a hard-constraint local trajectory optimizer. Each module is designed to handle **dynamic unknown obstacles**, which requires planning in spatiotemporal space, adapting trajectories as obstacles move unpredictably, and computing safe trajectories in real time.

SANDO processes point cloud data from a LiDAR sensor and/or a depth camera. The point cloud is processed by the map manager, which generates a voxel map with heat map costs for both static and dynamic obstacles. Point cloud data are also used by a dynamic obstacle tracker, where dynamic obstacles are detected, clustered, and tracked to estimate their current positions and predict their future trajectories, as detailed in Section VII.

The voxel map and predicted obstacle trajectories are provided as inputs to the heat map-based global planner and

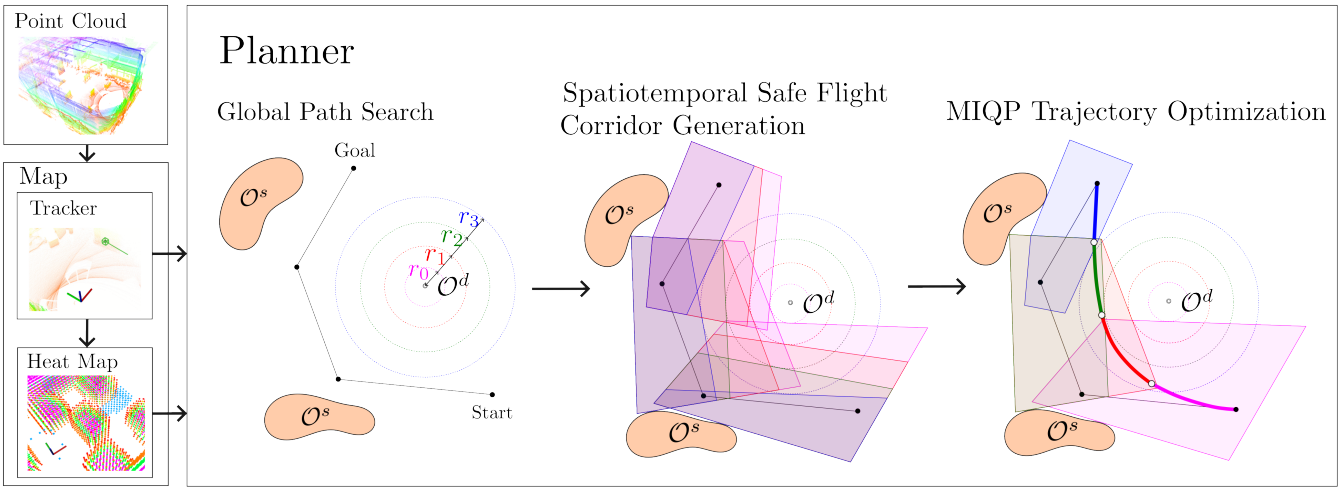


Fig. 2. SANDO system overview: Point cloud data are processed by the Map Manager, which detects static obstacles (denoted O^s) and dynamic obstacles (denoted O^d) and estimates the states of dynamic obstacles. The Dynamic Obstacle Tracker feeds its predictions to the Heat Map module, which generates soft costs around both static and dynamic obstacles (Section V). The heat map-based global planner computes a global path using A* search, and the predicted obstacle trajectories are used by the STSFC Generation module to produce time-layered polytope sequences (Section IV). These time-varying polytopes are then used in the Trajectory Optimization module to compute a safe trajectory (Section VI).

the STSFC Generation module. The global planner computes a global path from the agent's start position to a *subgoal* (a projection of the goal position on the local map around the agent) using heat map-based A* search with soft costs for both static and dynamic obstacles; see Section V for details. SANDO then constructs STSFCs, which account for spatial and temporal dimensions, by inflating dynamic obstacles based on their worst-case reachable sets at each time layer, as described in Section IV.

Since dynamic obstacles can change their motion unpredictably, fast trajectory optimization is needed. SANDO introduces a variable elimination approach that reduces the number of decision variables and constraints in the optimization problem, enabling fast hard-constraint optimization; see Section VI. The resulting trajectory is then sent to the low-level controller for execution. Replanning is triggered at 100 Hz; however, a new replanning cycle begins only after the previous one completes, so the effective replanning rate is bounded by the total replanning time (typically 20–35 ms in hardware, yielding an effective rate of approximately 30–50 Hz). If replanning fails to find a feasible trajectory, the agent continues executing the previously computed trajectory; if no valid trajectory remains, the agent hovers in place. Point cloud processing, dynamic obstacle tracking, map management, and SANDO's planning modules are all parallelized.

IV. Spatiotemporal Safe Flight Corridor (STSFC) Generation

We first define the key terminology used throughout the paper. Let t_0 denote the *current planning time*. The trajectory is composed of N *pieces* $\boldsymbol{x}_0, \ldots, \boldsymbol{x}_{N-1}$, each a cubic Bézier polynomial indexed by $n \in \{0, \ldots, N-1\}$. Each piece n executes during the *time interval* $[t_0 + n \cdot dt,\ t_0 + (n+1) \cdot dt]$, where dt is the uniform duration per piece, so that the full trajectory spans $[t_0,\ t_0 + N \cdot dt]$. The global path consists of P *path segments*, indexed by $p \in \{0, \ldots, P-1\}$. Each of the K tracked dynamic obstacles, indexed by $k \in \{1, \ldots, K\}$, occupies a region $O_k(t) \subset \mathbb{R}^3$ at time t, with true centroid $\mathbf{c}_k(t) \in \mathbb{R}^3$. The obstacle region is contained in an axis-aligned bounding box (AABB) centered at $\mathbf{c}_k(t)$ with half-extents $\mathbf{h}_k = (h_k^x,\ h_k^y,\ h_k^z) \in \mathbb{R}^3_{>0}$:

$$O_k(t) \ \subseteq\ \left\{\mathbf{x} \in \mathbb{R}^3 : |x_i - c_k^i(t)| \le h_k^i,\ \ i \in \{x, y, z\}\right\}.$$

In dynamic environments, safety corridors must account for both spatial constraints and how obstacles move over time. Traditional corridor generation methods, such as [2], [25], [26], produce purely spatial polytopes along a path, which is sufficient for static environments but does not capture how free space changes as obstacles move. This requires corridors that account for both the spatial location and the time at which each region is safe. To this end, we introduce *STSFCs*, time-varying polytope sequences where each time layer represents the collision-free region at a given duration in the trajectory's time horizon. This section describes the structure of STSFCs, the time allocation strategy, and the obstacle inflation method used to ensure safety guarantees. The global path input used during corridor generation is described in Section V.

A. Temporal Corridor Structure

An STSFC is represented as a two-dimensional array $C[n][p]$ where:

- $n \in \{0, \ldots, N-1\}$ indexes the *time layer*, representing discrete time intervals along the trajectory,
- $p \in \{0, \ldots, P-1\}$ indexes the *spatial polytope* within each time layer, representing free-space regions at that time.

Each element $C[n][p]$ is a convex polytope defined by linear constraints $\{\mathbf{F}_{np}, \mathbf{g}_{np}\}$ such that a point $\mathbf{x} \in \mathbb{R}^3$ is inside the polytope if $\mathbf{F}_{np}\mathbf{x} \le \mathbf{g}_{np}$. Each time layer n corresponds to the time interval $[t_0 + n \cdot dt,\ t_0 + (n+1) \cdot dt]$, matching the time interval of trajectory piece n. This structure allows the trajectory optimizer to select different

spatial corridors at different times, adapting to the evolving obstacle configuration. Note that we generate a polytope for every combination of time layer n and path segment p, even though some assignments may seem unlikely (e.g., the last time layer in the first path segment, or the first time layer in the last path segment). This is necessary because the MIQP solver determines the piece-to-polytope assignment, and the assignment of pieces is not known a priori. Moreover, as reported in Tables IX and XI, the total STSFC generation time for 10–15 polytopes is only a few milliseconds, so generating polytopes for all combinations adds negligible computational cost.

B. Obstacle Inflation by Reachable Radius

For each dynamic obstacle k with estimated position $\hat{\mathbf{c}}_k(t_0) \in \mathbb{R}^3$ and each time layer n, we compute the worst-case reachable position set by inflating the obstacle's AABB half-extents by the per-layer reachable radius:

$$r_n = v_{\max}^{\text{obs}} \cdot (n{+}1) \cdot dt + \epsilon,$$

where $(n{+}1) \cdot dt$ is the end time of layer n (we use the end time rather than the midpoint to ensure that the inflation covers the worst-case obstacle displacement over the *entire* layer), $v_{\max}^{\text{obs}}$ is the maximum obstacle velocity, and $\epsilon \geq 0$ is a bound on the per-axis position estimation error from the obstacle tracker (Section VII). Because r_n grows with n, obstacles are inflated more in later time layers, reflecting the increasing uncertainty in obstacle position over time.

The inflated obstacle region for time layer n is the Minkowski sum of the obstacle's estimated AABB with an axis-aligned cube of half-side r_n:

$$\hat{O}_k^n := \left\{\mathbf{x} \in \mathbb{R}^3 : |x_i - \hat{c}_k^i(t_0)| \leq h_k^i + r_n, \;\; i \in \{x, y, z\}\right\},$$

yielding an enlarged AABB with half-extents $\mathbf{h}_k + r_n\mathbf{1}$ centered at $\hat{\mathbf{c}}_k(t_0)$. In addition to r_n, a fixed safety margin r_{margin} (see Table III) is added to the AABB half-extents $\mathbf{h}_k$ during obstacle detection (Section VII), providing an additional buffer that absorbs position estimation error and controller tracking error beyond the reachable-set inflation. In practice, this margin allows setting $\epsilon = 0$ in r_n while still maintaining robustness to estimation and tracking errors, since r_{margin} serves the same role as a nonzero ϵ (see Section VIII). The inflated AABB voxels are then used to generate STSFCs for time layer n.

C. Unknown Space Inflation

SANDO maintains a voxel map that is built incrementally from sensor observations: each voxel is classified as free, occupied, or unknown, where unknown voxels are those that have not yet been observed by the sensor. In such environments, dynamic obstacles may emerge from unknown regions at any time. If these regions are treated as free during corridor generation, the resulting corridors may overlap with space that is actually occupied, violating safety. To address this, SANDO inflates the boundaries of unknown (unobserved) voxels, treating them as occupied during the ellipsoid-based corridor decomposition.

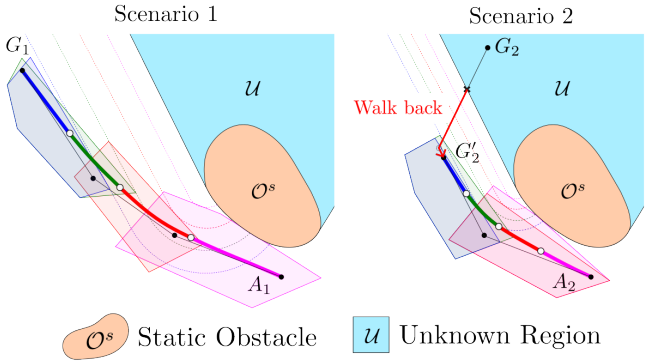


Fig. 3. Unknown-space inflation for STSFC corridor generation. Boundary voxels of unknown regions are inflated using the same per-layer reachable radius r_n as dynamic obstacles. **Scenario 1:** The goal G_1 lies outside the unknown region. The trajectory from A_1 to G_1 stays outside the inflated boundary, guaranteeing safety against unobserved dynamic obstacles. **Scenario 2:** The goal G_2 lies inside the unknown region. SANDO walks back along the global path from the unknown boundary until a point outside the worst-case inflated region is found, yielding a new subgoal G_2'. Since the trajectory to G_2' is shorter, the per-layer r_n values are smaller compared to Scenario 1, producing less conservative inflation and larger corridors. For simplicity, the figure does not show the inflation on the right side of the unknown region, which is also performed in practice. The same inflation process is applied to all unknown boundaries, ensuring safety regardless of the global path's direction.

Specifically, let $\mathcal{U}(t_0) \subset \mathbb{R}^3$ denote the set of unknown voxels at planning time t_0, and let $\mathcal{B}_{\mathcal{U}}(t_0)$ denote its boundary voxels. Each boundary voxel is inflated by the same per-layer radius r_n used for dynamic obstacles. The inflated unknown region for time layer n is:

$$\hat{\mathcal{U}}^n := \mathcal{U}(t_0) \;\cup\; \big(\mathcal{B}_{\mathcal{U}}(t_0) \oplus B_\infty(r_n)\big),$$

where $B_\infty(r_n)$ is the L_∞-ball of radius r_n and $\oplus$ denotes the Minkowski sum. The inflated unknown voxels are included in the obstacle set used for corridor generation, ensuring that any dynamic obstacles that could emerge from these regions are accounted for in the resulting polytopes $C[n][p]$ (see Fig. 3). When the goal lies outside the unknown region (Scenario 1 in Fig. 3), the trajectory stays entirely in observed space and the inflated boundary provides a safety buffer against unobserved obstacles.

a) Safe subgoal selection: When the global path enters the unknown region, corridors cannot be generated beyond the unknown boundary. To handle this, SANDO finds the first point where the global path intersects the unknown boundary and walks back along the path until it reaches a point outside the worst-case inflated unknown region, yielding a new subgoal G' (Scenario 2 in Fig. 3). The worst-case inflation is computed from the planning horizon, the agent's dynamic constraints, and the maximum time allocation factor (see Section VI-A.2) in a given replanning cycle. Since the trajectory to G' is shorter than to the original goal, each per-layer r_n is smaller, producing less conservative inflation and larger corridors. Note that this is a heuristic: if the global path runs close to the unknown boundary, the walk-back may not find a subgoal fully outside the worst-case inflated region. In practice, however, the reduced trajectory duration sufficiently shrinks r_n to yield feasible corridors.

The resulting STSFCs are used as constraints in the MIQP trajectory optimization described in Section VI. System-level parameters, including r_{drone}, $v_{\max}^{\text{obs}}$, and the replanning rate, are configured per-environment and listed in Section IX.

V. Heat Map-Based Global Planner

As described in Section IV, STSFC corridors shrink as dynamic obstacles are inflated by their worst-case reachable sets. If the global path passes close to dynamic obstacles, the resulting corridors may become too small for the trajectory optimizer to find a feasible solution. A naive approach would be to hard-block the entire worst-case reachable set as occupied in the planner's occupancy grid. However, this creates a critical replanning failure mode: at the next replanning step, the agent's current position may fall inside the previously blocked region, since the obstacle has moved and the reachable set has shifted, making the new planning query infeasible (see *No Heat Map & No STSFC* approach at $t = t_1$ in Fig. 4).

To avoid this, SANDO employs a heat map-based A* planner that uses soft costs rather than hard occupancy constraints for dynamic obstacles. The heat map proactively steers the global path away from regions where corridors will shrink, without blocking those regions entirely. This ensures that the global path search always has a feasible start configuration, while still biasing the path toward regions with larger corridors. Static and dynamic obstacles are both incorporated through heat maps, and only known static obstacles and the current location of dynamic obstacles enforce hard infeasibility. These occupied voxels are inflated by the drone radius r_{drone} (the circumscribed radius of the vehicle) to account for the agent's physical extent. Unknown space is treated as free in the global planner to allow the agent to plan paths toward unexplored regions. In contrast, during STSFC generation, unknown space is treated as occupied, and when unknown-space inflation is enabled, the boundaries of unknown regions are further inflated by the per-layer reachable radius r_n so that the resulting corridors account for obstacles that may emerge from unobserved space (see Section IV-C).

A. Static Obstacle Heat Map

We define a static heat map $H^s : \mathbf{q} \in \mathbb{R}^3 \to \mathbb{R}_{\geq 0}$ using a distance-based cost that decreases with distance from obstacle surfaces. Only boundary voxels (surface voxels of obstacles) serve as heat sources. For each boundary voxel b with center $\mathbf{c}_b$ and halo radius R^s (the maximum distance at which the voxel influences the cost), the heat contribution at a query point $\mathbf{q} \in \mathbb{R}^3$ is:

$$H_b^s(\mathbf{q}) = \begin{cases} \alpha^s \left(1 - \dfrac{\|\mathbf{q} - \mathbf{c}_b\|}{R^s}\right)^{p^s}, & \|\mathbf{q} - \mathbf{c}_b\| \leq R^s, \\ 0, & \text{otherwise}, \end{cases}$$

where α^s is the intensity scale and p^s controls the decay shape. The aggregate static heat is $H^s(\mathbf{q}) = \min(\max_b H_b^s(\mathbf{q}), H_{\max})$, using max aggregation to avoid artificially inflating costs in dense obstacle regions, capped at $H_{\max}$ (the maximum allowable heat value). See Table II for the static heat parameters used in our experiments.

B. Dynamic Obstacle Heat Map

For each tracked dynamic obstacle k with estimated position $\hat{\mathbf{c}}_k$, AABB half-extents $\mathbf{h}_k$, and predicted trajectory $\mu_k(t)$ from the obstacle tracker (Section VII) over a prediction time horizon T_h, we construct a per-obstacle heat $H_k(\mathbf{q})$ combining a penalty around the current position and a penalty around the predicted trajectory tube.

a) Base heat around the current position: We define the base obstacle radius as $R_{0,k} \coloneqq \max_i h_k^i + r_{\text{margin}}$, where r_{margin} is a fixed safety margin (see Table III), and the horizon-limited reachable radius as $R_k^d \coloneqq R_{0,k} + v_{\max}^{\text{obs}} T_h$. The base heat is:

$$H_k^{\text{base}}(\mathbf{q}) = \begin{cases} \alpha_0^d \left(1 - \dfrac{\|\mathbf{q} - \hat{\mathbf{c}}_k\|}{R_k^d}\right)^{p^d}, & \|\mathbf{q} - \hat{\mathbf{c}}_k\| \leq R_k^d, \\ 0, & \text{otherwise}, \end{cases}$$

where α_0^d is the base cost scale and p^d controls the decay shape.

b) Tube penalty from predicted motion: We discretize the horizon into M_{tube} samples $\{t_j\}_{j=0}^{M_{\text{tube}}-1}$ with predicted centers $\hat{\mathbf{c}}_{k,j} = \mu_k(t_j)$. The tube radius $R_{k,j} \coloneqq R_{0,k} + \gamma_k t_j$ grows with prediction time to account for increasing uncertainty, where γ_k is the per-obstacle uncertainty growth rate. The tube penalty is:

$$H_k^{\text{tube}}(\mathbf{q}) = \alpha_1^d \max_j \left(w(t_j) \left[\max\left(0,\, 1 - \frac{\|\mathbf{q} - \hat{\mathbf{c}}_{k,j}\|}{R_{k,j}}\right) \right]^{q^d} \right),$$

where $w(t) = \exp(-t/\tau^d)$ with $\tau^d \coloneqq \tau_{\text{ratio}}^d \cdot T_h$ is an exponential time weight that discounts predictions further into the future (since they are less reliable), τ_{ratio}^d is a scaling factor that sets the decay time constant as a fraction of the horizon, α_1^d scales the tube penalty, and q^d controls how sharply the cost increases near the predicted path. The tube radius $R_{k,j}$ grows with time while the time weight $w(t_j)$ decays: near-future predictions receive high weight over a small region, whereas far-future predictions receive low weight over a larger region. This is a heuristic that balances spatial coverage against prediction reliability, and we found empirically that it produces effective obstacle avoidance paths across a wide range of environments. The total per-obstacle heat combines both components: $H_k(\mathbf{q}) = H_k^{\text{base}}(\mathbf{q}) + H_k^{\text{tube}}(\mathbf{q})$, where H_k^{base} penalizes proximity to the obstacle's current position and H_k^{tube} penalizes proximity to its predicted future path. The dynamic heat across all obstacles is $H^d(\mathbf{q}) = \max_k H_k(\mathbf{q})$, using max aggregation so that the most dangerous obstacle dominates the cost.

C. Combined Heat Formulation

Fig. 5 illustrates the combined heat map in a hardware experiment. The combined heat map merges the static and dynamic components via max aggregation:

$$H(\mathbf{q}) = \min\Big(\max\Big(H^s(\mathbf{q}),\, H^d(\mathbf{q})\Big),\, H_{\max}\Big),$$

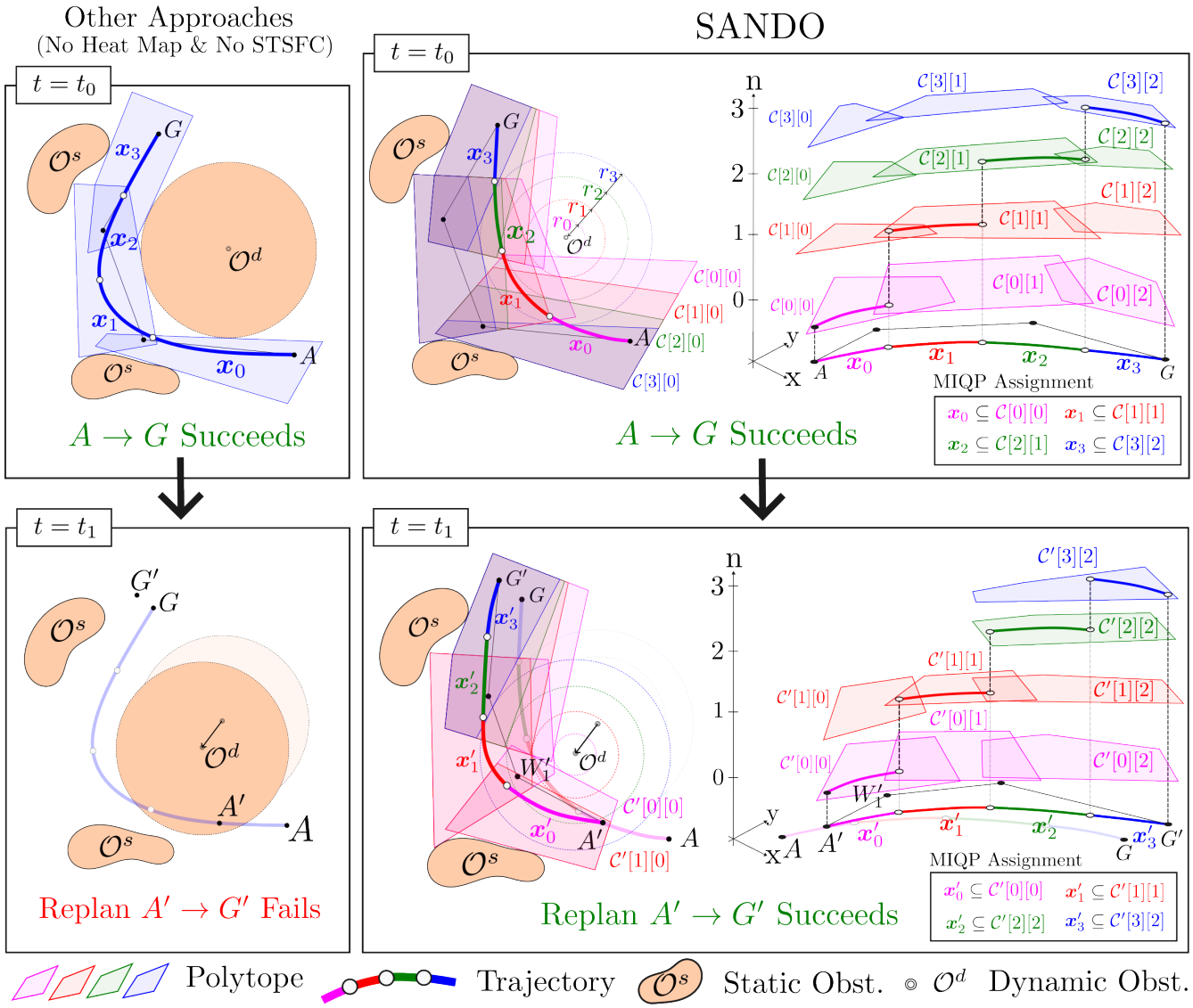


Fig. 4. STSFC corridors and MIQP trajectory optimization in dynamic environments. For visualization, the figure shows a 2D (x-y) case with the time-layer axis as the third dimension; in practice, SANDO uses full 3D (x, y, z) STSFCs. **Left column, other approaches (hard-blocking reachable sets):** At $t = t_0$ (top left), the planner hard-blocks the worst-case reachable set of $\mathcal{O}^d$ as occupied and computes a conservative global path $A \to G$ that avoids the entire blocked region. Although planning succeeds, the resulting trajectory is overly conservative. At $t = t_1$ (bottom left), the agent replans from the new position A' toward G', but A' now falls inside the shifted blocked region, making the query $A' \to G'$ infeasible. **Right column, SANDO (heat map-based soft costs):** At $t = t_0$, the 2D view (top center) shows the global path $A \to G$ steered away from $\mathcal{O}^d$ by soft heat costs, producing a less conservative trajectory. The temporal view (top right) visualizes the STSFC corridors $C[n][p]$ along the time-layer axis, with increasing per-layer inflation radii r_n (see Section IV); the MIQP assigns each trajectory piece to a corridor ($\boldsymbol{x}_0 \subseteq C[0][0]$, $\boldsymbol{x}_1 \subseteq C[1][1]$, $\boldsymbol{x}_2 \subseteq C[2][1]$, and $\boldsymbol{x}_3 \subseteq C[3][2]$). At $t = t_1$, the 2D view (bottom center) and temporal view (bottom right) show successful replanning from A' to G': the updated corridors $C'[n][p]$ remain valid and the MIQP assignment ($\boldsymbol{x}'_0 \subseteq C'[0][0]$, $\boldsymbol{x}'_1 \subseteq C'[1][1]$, $\boldsymbol{x}'_2 \subseteq C'[2][2]$, and $\boldsymbol{x}'_3 \subseteq C'[3][2]$) succeeds, since soft costs never blocked the region around A'. Even though in the first global path section ($A' \to W_1$), the corridor generation for $n = 2$ (green) and $n = 3$ (blue) fails to produce corridors, since $n = 0$ (pink) and $n = 1$ (red) still have valid corridors, the MIQP can find a feasible trajectory that safely navigates through the dynamic environment. (The dynamic obstacle is depicted as a point mass for visual clarity; in practice, obstacles have finite AABB extents $\mathbf{h}_k$, and the reachable set is the Minkowski sum of the AABB with a cube of half-side r_n.)

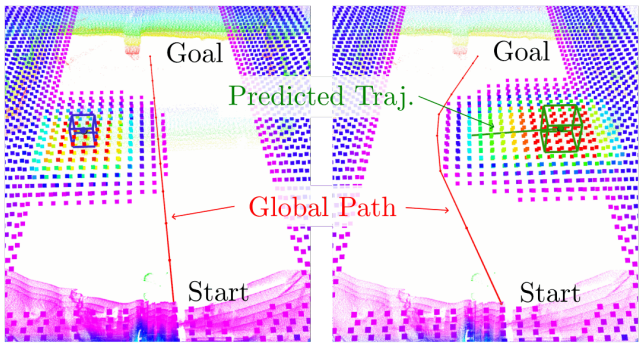


Fig. 5. Visualization of the heat map-based global planner in hardware. Static and dynamic obstacle heat maps are combined via max aggregation, and the A* planner computes a global path that avoids high-cost regions while maintaining feasibility. From the wall on the side, the static heat decays with distance, while from the dynamic obstacle (blue and green boxes), the heat decays both spatially and temporally. On the left, the dynamic obstacle is not moving, so the heat is dominated by the base heat around its current position. On the right, the dynamic obstacle is moving toward the center of the room (the prediction is visualized as the green line), so the tube penalty creates a heat trail along the predicted path, steering the global path away from that region.

where $H_{\max}$ caps the total heat. Max aggregation avoids double-penalization and ensures the highest risk dominates.

D. A* with Hard Occupancy and Soft Cost Penalties

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote the 26-connected voxel graph (i.e., each voxel is connected to all $3^3 - 1 = 26$ neighbors in its $3\times 3\times 3$ neighborhood), where $\mathcal{V}$ is the set of free voxel centers and $\mathcal{E}$ contains edges between neighboring free voxels. For an edge $(\mathbf{q}_i, \mathbf{q}_{i+1}) \in \mathcal{E}$, the edge cost is:

$$c(\mathbf{q}_i, \mathbf{q}_{i+1}) \;=\; d(\mathbf{q}_i, \mathbf{q}_{i+1}) \;+\; w_{\text{heat}}\, H(\mathbf{q}_{i+1}),$$

where $d(\mathbf{q}_i, \mathbf{q}_{i+1})$ is the Euclidean distance between neighboring voxel centers and $w_{\text{heat}} > 0$ is a tunable weight that balances the heat penalty against path length. A* searches for the minimum-cost path from start $\mathbf{q}_s$ to goal $\mathbf{q}_g$ using the goal-directed heuristic $h(\mathbf{q}) = \|\mathbf{q} - \mathbf{q}_g\|$, yielding a global path in known-free space that avoids regions near both static and dynamic obstacles.

a) Heat map parameter selection: The heat map parameters (listed in Table II) influence only the global path quality, not the safety guarantee, which is enforced by the hard STSFC constraints in the trajectory optimizer. A path that passes too close to obstacles may result in small corridors and reduced optimization feasibility, while overly aggressive heat penalties can produce unnecessarily long detours. In practice, we found the same parameter set (Table II) to be effective across all simulation environments without per-scenario tuning. For the hardware experiments, we increased w_{heat} from 5.0 to 20.0 in the five-obstacle configuration (Experiments 11–16) to account for the smaller physical environment (8×20 m vs. 100×40 m in simulation), where obstacles occupy a proportionally larger fraction of the space and stronger steering is needed to maintain corridor feasibility.

VI. Trajectory Optimization

Building on the MIQP framework for static environments [2], [19], [27], SANDO optimizes position trajectories using hard-constraint Mixed-Integer Quadratic Programming (MIQP) extended to dynamic settings with time-varying corridors. Although MIQP introduces binary variables for piece-polytope assignment and increases computational cost relative to soft-constraint methods, it guarantees collision-free trajectories. To reduce complexity, we leverage a variable elimination technique [28] that reduces the number of decision variables by symbolically solving the linear equality constraints, similar to the closed-form reductions in [29] but extended to the MIQP setting (see Section VI-A.1 for details). SANDO also incorporates a parallelized time allocation strategy that launches multiple MIQP instances with different time allocations concurrently, selecting the best solution that satisfies all constraints (see Section VI-A.2 for details).

A. Trajectory Optimization

We model the agent with triple integrator dynamics and state vector $\mathbf{x}^T = \begin{bmatrix}\boldsymbol{x}^T & \boldsymbol{v}^T & \boldsymbol{a}^T\end{bmatrix}$, where $\boldsymbol{x}$, $\boldsymbol{v}$, and $\boldsymbol{a}$ denote position, velocity, and acceleration, respectively.

We formulate trajectory optimization using an N-piece composite Bézier curve with P spatial polytopes per time layer. As described in Section IV, STSFCs in dynamic environments are time-layered, with polytopes indexed by both time layer and spatial location. We reuse $n \in \{0 : N-1\}$ and $p \in \{0{:}P-1\}$ from Section IV: since each trajectory piece n executes during time layer n of the STSFC, the same index identifies both the piece and its corresponding time layer. Similarly, p indexes the spatial polytope within that layer. The time interval dt per piece is uniform and matches the STSFC layer duration. Fig. 4 illustrates this process: the MIQP assigns each trajectory piece to a spatial polytope within its time layer (e.g., $\boldsymbol{x}_0 \subseteq C[0][0]$, $\boldsymbol{x}_1 \subseteq C[1][1]$), ensuring the trajectory remains in free space both spatially and temporally.

The control input, jerk, remains constant within each piece, allowing the position trajectory of each piece to be represented as a cubic polynomial. Note that this implies jerk is discontinuous at piece boundaries; however, cubic polynomial representations are widely adopted in real-time planning [1], [2], [30]:

$$\boldsymbol{x}_n(\tau) = \boldsymbol{a}_n\tau^3 + \boldsymbol{b}_n\tau^2 + \boldsymbol{c}_n\tau + \boldsymbol{d}_n, \quad \tau \in [0, dt] \qquad (1)$$

where $\boldsymbol{a}_n, \boldsymbol{b}_n, \boldsymbol{c}_n, \boldsymbol{d}_n \in \mathbb{R}^3$ are the coefficients of the cubic spline in piece n.

We now discuss the constraints for the optimization formulation. Continuity constraints are added between adjacent pieces to ensure the trajectory is continuous in position, velocity, and acceleration:

$$\mathbf{x}_{n+1}(0) = \mathbf{x}_n(dt) \quad \text{for} \quad n \in \{0{:}N-2\} \qquad (2)$$

The Bézier curve control points $\boldsymbol{p}_{nj}$ ($j \in \{0{:}3\}$) associated

with each piece n are:

$$\begin{aligned} \boldsymbol{p}_{n0} &= \boldsymbol{d}_n, \quad \boldsymbol{p}_{n1} = \frac{\boldsymbol{c}_n dt + 3\boldsymbol{d}_n}{3} \\ \boldsymbol{p}_{n2} &= \frac{\boldsymbol{b}_n dt^2 + 2\boldsymbol{c}_n dt + 3\boldsymbol{d}_n}{3} \\ \boldsymbol{p}_{n3} &= \boldsymbol{a}_n dt^3 + \boldsymbol{b}_n dt^2 + \boldsymbol{c}_n dt + \boldsymbol{d}_n \end{aligned} \tag{3}$$

Since a Bézier curve lies within the convex hull of its control points [31], constraining all control points to lie inside a convex polytope guarantees that the entire piece remains inside that polytope. To assign pieces to polytopes, we introduce binary variables z_{np}, where $z_{np} = 1$ if piece n is assigned to polytope p, and $z_{np} = 0$ otherwise. This condition is enforced through the following constraint:

$$z_{np} = 1 \implies \mathbf{F}_{np}\boldsymbol{p}_{nj} \leq \mathbf{g}_{np}, \text{ for } \quad j \in \{0{:}3\}, \forall n, \forall p \tag{4}$$

where polytopes are time-layered as described in Section IV, with $(\mathbf{F}_{np}, \mathbf{g}_{np})$ denoting the polytope at time layer corresponding to piece n and spatial index p. In dynamic environments, the polytope constraints $\mathbf{F}_{np}$ and $\mathbf{g}_{np}$ vary with both the trajectory piece n (time) and spatial polytope index p, reflecting the temporal evolution of free space as obstacles move. Each piece must be assigned to at least one polytope, which is ensured by the constraint:

$$\sum_{p=0}^{P-1} z_{np} \geq 1, \quad \forall n \tag{5}$$

To ensure the trajectory starts at the initial state and ends at the final state, we impose the following constraints:

$$\mathbf{x}_0(0) = \mathbf{x}_{\text{init}}, \qquad \mathbf{x}_{N-1}(dt) = \mathbf{x}_{\text{final}} \tag{6}$$

where $\mathbf{x}_{\text{init}}$ is the initial state, and $\mathbf{x}_{\text{final}}$ is the final state. The final state is set to the $(P{+}1)$-th waypoint on the global path with zero velocity and zero acceleration, so that the trajectory spans exactly P path segments and stops at the $(P{+}1)$-th waypoint. Since SANDO replans in a receding horizon manner, only the initial portion of the trajectory is typically executed before a new plan is computed; the agent does not necessarily reach the final stop state of each optimized trajectory.

For dynamic constraints, we define the velocity control points $\boldsymbol{v}_{nj}$ ($j \in \{0{:}2\}$), acceleration control points $\boldsymbol{a}_{nj}$ ($j \in \{0{:}1\}$), and jerk $\boldsymbol{j}_n$ for each piece n. By the convex hull property of Bézier curves, bounding these control points guarantees that the continuous velocity, acceleration, and jerk remain within limits throughout each piece:

$$\begin{aligned} \left\|\boldsymbol{v}_{nj}\right\|_\infty &\leq v_{\max}, \quad j \in \{0{:}2\} \\ \left\|\boldsymbol{a}_{nj}\right\|_\infty &\leq a_{\max}, \quad j \in \{0{:}1\} \\ \left\|\boldsymbol{j}_n\right\|_\infty &\leq j_{\max} \end{aligned} \tag{7}$$

where $v_{\max}$, $a_{\max}$, and $j_{\max}$ denote the maximum allowable velocity, acceleration, and jerk, respectively. The objective function penalizes the squared jerk along the trajectory for smooth motion:

$$J = \sum_{n=0}^{N-1} \left\|\boldsymbol{j}_n\right\|^2 .$$

The complete MIQP problem is then formulated as:

$$\begin{aligned} \min_{\boldsymbol{a}_n, \boldsymbol{b}_n, \boldsymbol{c}_n, \boldsymbol{d}_n, z_{np}} \quad & J \\ \text{s.t.} \quad & \text{Eqs. (2), (4), (5), (6), and (7)} \end{aligned} \tag{8}$$

1) Variable Elimination: In the MIQP formulation of Eq. (8), each piece n introduces four coefficient vectors ($\boldsymbol{a}_n$, $\boldsymbol{b}_n$, $\boldsymbol{c}_n$, $\boldsymbol{d}_n$), giving $4N$ decision variables per axis (x, y, z), i.e., $12N$ in 3D. However, the continuity constraints (Eq. (2)) and boundary conditions (Eq. (6)) impose $3N + 3$ equality constraints per axis: $3(N{-}1)$ from position, velocity, and acceleration continuity at the $N{-}1$ interior piece boundaries, plus 6 from the initial and final boundary conditions (position, velocity, and acceleration at start and end). By symbolically solving these equality constraints, we can express most coefficients as affine functions of a small set of remaining variables, which (1) reduces the number of decision variables to $N - 3$ per axis and (2) removes all equality constraints from the optimization.

For instance, with $N = 4$ pieces there are $4N = 16$ variables and $3N + 3 = 15$ equality constraints per axis, leaving only one decision variable per axis. Symbolic elimination reveals this remaining variable to be $\boldsymbol{d}_3$ (the first control point of the final piece); all other coefficients become affine functions of $\boldsymbol{d}_3$. In general, the number of remaining decision variables per axis is $4N - (3N + 3) = N - 3$; for example, $N = 5$ yields 2 and $N = 6$ yields 3 per axis, with the same elimination procedure applied. However, the symbolic expressions grow combinatorially with N: each remaining variable's affine coefficients depend on all boundary conditions and continuity relations, so the closed-form expressions become prohibitively large for $N > 7$. In our implementation, we precompute the symbolic elimination offline for $N \in \{4, 5, 6, 7\}$, which covers the operating range used in all experiments.

This leads to a revised MIQP with many fewer decision variables and no equality constraints:

$$\begin{aligned} \min_{\boldsymbol{d}_3, z_{np}} \quad & J \\ \text{s.t.} \quad & \text{Eqs. (4), (5), and (7).} \end{aligned}$$

This MIQP is solved using Gurobi [32]. Section IX-B performs an ablation study to evaluate the computational benefits of this variable elimination technique, and it demonstrates up to 7.4× reduction in optimization time.

2) Time Allocation and Parallelization: The time allocated to each trajectory piece strongly affects feasibility and optimality: too short and the dynamical limits are violated, too long and the trajectory is overly slow. Following [2], SANDO computes a baseline time per piece dt_0 from the per-axis minimum-time solutions under velocity, acceleration, and jerk limits, and scales it by a factor $f \geq 1$ to obtain the actual time per piece $dt = f \cdot dt_0$.

To search over time allocations efficiently, SANDO main-

tains a sliding window of M candidate factors $\{f_1, \ldots, f_M\}$ spanning a range of width 2κ centered on a mean value, with uniform step size Δf, where $M = \lfloor 2\kappa/\Delta f \rfloor + 1$. At each replanning iteration, M MIQP solver threads are launched in parallel, one per factor, each with $dt = f_i \cdot dt_0$. Since each factor yields a different dt, a separate STSFC is generated per thread to reflect the corresponding obstacle inflation radii; the per-thread STSFC computation cost is reported in Sections IX and X. As soon as any thread finds a feasible solution, the remaining threads are terminated and that solution is used.

The factor window adapts for the next replanning cycle based on the outcome:

- **Success:** The window is recentered so that the successful factor becomes the median, biasing the next iteration toward similar time allocations.
- **All fail:** The window is shifted upward by one step Δf, increasing the time allocation to improve feasibility. If the window reaches a configurable upper bound $f_{\max}$ (see Table III), it resets to the initial position.

VII. Dynamic Obstacle Detection and Tracking

SANDO detects and tracks dynamic obstacles from raw point cloud data through a multi-stage pipeline.

A. Detection via Temporal Occupancy Grid

Inspired by Dynablox [33], SANDO constructs a temporal occupancy grid to classify voxels as static or dynamic. Voxels that remain occupied beyond a configurable duration are classified as static. When a voxel transitions from free to occupied and has fewer than a threshold number of static neighbors, it is classified as dynamic, indicating a newly appearing moving object rather than part of an existing static structure. Dynamic labels persist for a configurable duration to maintain temporal continuity during brief sensor occlusions. False positives are mitigated by the static-neighbor threshold: voxels adjacent to many static voxels are not classified as dynamic, preventing edges of static structures from being misidentified as moving objects.

B. Clustering and Data Association

Dynamic voxels are grouped into clusters using Euclidean clustering. For each cluster, the centroid and AABB half-extents are computed. Since onboard sensors typically observe only one face of an obstacle, the AABB is inflated to a cubic shape using the largest observed dimension, providing a conservative size estimate for collision avoidance. Clusters are associated with existing tracks via nearest-neighbor matching within a distance threshold; unmatched clusters initialize new tracks. Nearest-neighbor association can produce incorrect matches when obstacle trajectories cross or obstacles are closely spaced; however, the tracker is a modular component and can be replaced with more sophisticated methods (e.g., the Hungarian algorithm) without changing the planning framework.

C. Adaptive Extended Kalman Filter (AEKF)

Each tracked obstacle k is modeled with a 9-state vector $\mathbf{x}_k = [\mathbf{p}_k^\top, \mathbf{v}_k^\top, \mathbf{a}_k^\top]^\top$, where $\mathbf{p}_k$, $\mathbf{v}_k$, and $\mathbf{a}_k$ denote the obstacle's position, velocity, and acceleration, using a constant-acceleration process model. The measurement is the cluster centroid (position only). To handle unknown and time-varying noise characteristics, we employ an AEKF [34] that continuously updates both the measurement noise covariance R_i and the process noise covariance Q_i at each filter step i using exponential forgetting with factor α:

$$R_i = \alpha R_{i-1} + (1-\alpha)\,\epsilon_i \epsilon_i^\top, \tag{9}$$

$$Q_i = \alpha Q_{i-1} + (1-\alpha)\,K_i d_i d_i^\top K_i^\top, \tag{10}$$

where ϵ_i is the residual, d_i is the innovation, and K_i is the Kalman gain. When a new obstacle is detected, its initial Q_0 and R_0 are set to the average of the covariances from existing tracks, allowing the filter to use prior knowledge of the environment's noise characteristics.

D. Prediction and Output

Future positions are predicted using a constant-velocity model with the AEKF-estimated velocity. Although the AEKF uses a constant-acceleration process model for state estimation, we use constant-velocity for prediction because acceleration estimates are noisy and change rapidly, making them unreliable over longer prediction horizons; constant-velocity extrapolation is more robust in practice. For each tracked obstacle, the system publishes the predicted trajectory and AABB half-extents. Tracks that are not updated for a configurable timeout are deleted, allowing the system to discard obstacles that have left the sensor's field of view or stopped moving. The predicted trajectories and reachable sets are then used by the heat map-based global planner (Section V) and STSFC generator (Section IV).

VIII. Safety Analysis

SANDO provides formal collision-free guarantees through its STSFC framework. Using the notation introduced in Section IV, we recall that the per-layer inflation radius is:

$$r_n \coloneqq v_{\max}^{\text{obs}} \cdot (n+1) \cdot dt \; + \; \epsilon, \tag{11}$$

where the first term accounts for worst-case obstacle displacement from t_0 to $t_0 + (n{+}1) \cdot dt$, and ϵ accounts for the position estimation error (see Assumption 2).

A. Assumptions

Assumption 1 (Bounded obstacle velocity) There exists a known constant $v_{\max}^{\text{obs}} > 0$ such that:

$$\|\dot{\mathbf{c}}_k(t)\|_\infty \le v_{\max}^{\text{obs}}, \quad \forall\, k,\ \forall\, t \ge 0.$$

That is, each component of the obstacle's velocity is bounded: $|\dot{c}_k^i(t)| \le v_{\max}^{\text{obs}}$ for $i \in \{x, y, z\}$.

Assumption 2 (Bounded position estimation error) The obstacle tracker provides an estimated position $\hat{\mathbf{c}}_k(t_0)$ of each

obstacle at the planning time t_0. There exists a known constant $\epsilon \geq 0$ such that:

$$\|\mathbf{c}_k(t_0) - \hat{\mathbf{c}}_k(t_0)\|_\infty \leq \epsilon, \quad \forall\, k.$$

That is, the per-axis estimation error is bounded by ϵ.

Assumption 3 (Perfect trajectory tracking) The low-level controller tracks the planned trajectory exactly, i.e., the executed position coincides with the planned position at all times.

In practice, Assumptions 2 and 3 are not satisfied exactly. However, the safety margin r_{margin} (see Table III), which is added to each obstacle's AABB half-extents during detection, provides an additional buffer beyond the reachable-set inflation r_n. This margin absorbs both position estimation errors and controller tracking errors: when r_{margin} exceeds the combined worst-case estimation and tracking error, one can set $\epsilon = 0$ in Eq. (11) and still maintain the safety guarantee. In all experiments, we set $\epsilon = 0$ and rely on r_{margin} (0.1 m in simulation, 0.2 m in hardware) together with the drone radius r_{drone} to absorb these errors. The larger hardware margin accounts for the greater tracking errors observed on the physical platform.

Assumption 4 (Untracked obstacles in unknown space) Let $\mathcal{U}(t_0) \subset \mathbb{R}^3$ denote the set of unobserved (unknown) voxels at the planning time t_0. Any dynamic obstacle that is not currently tracked by the obstacle tracker is located entirely within $\mathcal{U}(t_0)$ at time t_0.

B. Corridor Construction

As described in Section IV, each dynamic obstacle k is inflated by r_n (Eq. (11)) to obtain $\hat{O}_k^n$, and when unknown-space inflation is enabled, the unknown region boundary is inflated by the same radius to obtain $\hat{\mathcal{U}}^n$ (Section IV-C). The corridor polytopes are then generated to be free of all inflated regions: For each time layer n and each spatial polytope p, the polytope $C[n][p]$ is generated to be free of all inflated tracked dynamic obstacles and, when unknown-space inflation is enabled, the inflated unknown region:

$$C[n][p] \cap \hat{O}_k^n = \emptyset, \quad \forall\, k,\ \forall\, n,\ \forall\, p, \tag{12}$$

$$C[n][p] \cap \hat{\mathcal{U}}^n = \emptyset, \quad \forall\, n,\ \forall\, p. \tag{13}$$

a) MIQP polytope assignment: As described in Section VI, the MIQP assigns each piece n to a polytope $C[n][p_n^*]$ such that all four control points lie inside (Eq. (4)). By the convex hull property of Bézier curves [31], the entire piece therefore remains inside $C[n][p_n^*]$.

C. Safety

Theorem 1 (Safety guarantee) Under Assumptions 1–3 and the corridor construction in Section VIII-B, the trajectory $\boldsymbol{x}(t)$ produced by the MIQP solver is collision-free with respect to all *tracked* dynamic obstacles for the duration of the trajectory. Furthermore, when unknown-space inflation is enabled (Eq. (13)) and Assumption 4 holds, the trajectory is also collision-free with respect to all *untracked* dynamic obstacles. That is:

$$\boldsymbol{x}(t) \notin O_k(t), \quad \forall\, t \in [t_0,\ t_0 + N \cdot dt],\ \forall\, k,$$

where k ranges over all tracked dynamic obstacles, and additionally over all untracked dynamic obstacles when unknown-space inflation is enabled.

Proof. Consider an arbitrary trajectory piece n executing over $[t_0 + n \cdot dt,\ t_0 + (n{+}1) \cdot dt]$ and any time $\bar{t}$ within that interval. Let p_n^* be the spatial polytope assigned to piece n by the MIQP (Eq. (4)). We proceed in three steps: first, we show the trajectory stays inside its assigned corridor polytope; second, we show the true obstacle remains inside its inflated region; third, we show the corridor and inflated region are disjoint, which together imply no collision.

(i) Trajectory is inside a corridor polytope. By Eq. (4), all four control points of piece n lie inside $C[n][p_n^*]$. Since a Bézier curve is contained within the convex hull of its control points [31], piece n remains inside $C[n][p_n^*]$ throughout its time interval, so $\boldsymbol{x}(\bar{t}) \in C[n][p_n^*]$.

(ii) The true obstacle is inside the inflated region. For tracked dynamic obstacles, from t_0 to $\bar{t}$ the true centroid can differ from the estimated position by at most ϵ (Assumption 2) plus $v_{\max}^{\text{obs}} \cdot (n{+}1) \cdot dt$ of motion (Assumption 1) per axis. Since $r_n = v_{\max}^{\text{obs}} \cdot (n{+}1) \cdot dt + \epsilon$, the true obstacle region is contained in the inflated AABB: $O_k(\bar{t}) \subseteq \hat{O}_k^n$. For untracked dynamic obstacles (when unknown-space inflation is enabled), by Assumption 4 the obstacle starts in $\mathcal{U}(t_0)$, and by Assumption 1 it moves at most $v_{\max}^{\text{obs}} \cdot (n{+}1) \cdot dt$ per axis during time layer n, so $O_k(\bar{t}) \subseteq \hat{\mathcal{U}}^n$.

(iii) Corridor and inflated regions are disjoint. By construction, the corridor polytope is disjoint from all inflated tracked dynamic obstacles (Eq. (12)): $C[n][p_n^*] \cap \hat{O}_k^n = \emptyset$. When unknown-space inflation is enabled, it is also disjoint from the inflated unknown region (Eq. (13)): $C[n][p_n^*] \cap \hat{\mathcal{U}}^n = \emptyset$.

Combining (i)–(iii): the trajectory lies inside the corridor, the obstacle lies inside its inflated region, and the two are disjoint, so $\boldsymbol{x}(\bar{t}) \notin O_k(\bar{t})$. Since $\bar{t}$ and k were arbitrary, the trajectory is collision-free with respect to all dynamic obstacles at all times. Safety with respect to static obstacles follows directly: the corridor polytopes are constructed in free space that excludes all static obstacles (inflated by the drone radius r_{drone}), so the trajectory cannot intersect any static obstacle either. □

a) Role of agent and obstacle velocities in the inflation radius.: The inflation radius $r_n = v_{\max}^{\text{obs}} \cdot (n{+}1) \cdot dt + \epsilon$ depends on both the obstacle velocity bound $v_{\max}^{\text{obs}}$ and the per-piece duration dt. As described in Section VI-A.2, $dt = f \cdot dt_0$, where dt_0 is the minimum feasible time per piece derived from the agent's dynamic constraints ($v_{\max}$, $a_{\max}$, $j_{\max}$) and $f \geq 1$ is the time allocation factor. Hence, tighter agent dynamic constraints reduce dt_0 and consequently dt, which shrinks r_n and produces larger corridors. Conversely, a higher $v_{\max}^{\text{obs}}$ increases r_n, requiring more conservative corridors and potentially reducing feasibility in dense environments.

D. When Assumptions Are Violated

The safety guarantee of Theorem 1 depends on both assumptions and the corridor construction properties.

Velocity bound violated: If an obstacle exceeds $v_{\max}^{\text{obs}}$ along any axis, its true position at time t can lie outside the inflated box $\hat{O}_k^n$. The corridor remains obstacle-free with respect to the *inflated* region, but the true obstacle may have moved into the corridor. As $v_{\max}^{\text{obs}}$ increases, the per-layer inflation radius r_n grows linearly, shrinking the STSFC corridors and eventually making trajectory optimization infeasible. For very fast obstacles, the planner would need to rely more heavily on trajectory prediction to tighten the inflation (inflating around the predicted future position rather than the worst-case reachable set from the current position), which is an avenue for future work.

Estimation error exceeded: The total buffer available to absorb estimation error is $\epsilon + r_{\text{margin}}$. In our experiments, $\epsilon = 0$ and $r_{\text{margin}} \in \{0.1, 0.2\}$ m, so safety is maintained as long as the per-axis estimation error does not exceed r_{margin}. If the actual error exceeds this margin, the inflated region may not fully contain the true obstacle, and collisions become possible. Users can increase either ϵ or r_{margin} to accommodate larger estimation errors at the cost of more conservative corridors.

Tracking error: If the low-level controller does not track the planned trajectory perfectly, the actual position may deviate from the planned position. Even though the planned trajectory lies inside the corridor, the actual trajectory may exit the corridor and potentially collide with obstacles. In practice, tracking errors are absorbed by the drone radius r_{drone} and the safety margin r_{margin}, which together provide a spatial buffer between the corridor boundary and the obstacle surface.

Unknown-space inflation disabled: When unknown-space inflation is disabled, the safety guarantee of Theorem 1 covers only tracked dynamic obstacles. Untracked dynamic obstacles emerging from unobserved space are not represented in the corridor generation, and the trajectory may enter regions where such obstacles are present.

Note that safety is guaranteed only within the local planning horizon $N \cdot dt$; long-term safety requires continuous replanning, as discussed in Section VIII-E.

E. Recursive Feasibility

Recursive feasibility means that if a feasible trajectory exists at the current replanning step, one will also exist at the next step. OA-MPC [35] and Stamouli et al. [13] guarantee this for MPC in dynamic environments using a *shrinking horizon* tied to a fixed mission duration T, so the remaining planning time decreases at each step. Because the horizon shrinks toward the same end time T, the obstacle's reachable sets do not grow between replanning steps, and the previous plan remains feasible for the next step. However, this strategy is not suitable for long-duration navigation where the goal may be far from the agent and the mission duration is not fixed.

SANDO could adopt the same strategy if the goal were close enough to be reached within a single planning horizon, but this is not generally the case for long-range navigation. As described in Section III, SANDO replans in a receding horizon manner toward a subgoal that moves with the agent. Since the subgoal moves with the agent and the horizon does not shrink toward a fixed end time, the assumptions required for recursive feasibility do not hold. One alternative is to adopt the approach of Wu et al. [36] when the agent reaches a subgoal but cannot find a new trajectory. Ref. [36] guarantees infinite-horizon safety by allowing the agent to fly away from obstacles, but this requires the agent to be faster than all obstacles and the environment to be open, which is unrealistic in cluttered settings.

Theorem 1 guarantees collision-free execution within each individual planning horizon, but SANDO generally does not guarantee that a feasible plan will exist at every future replanning step due to the receding horizon nature where we use a different subgoal at each step. However, as shown in simulations in Section IX and hardware experiments in Section X, SANDO often finds new trajectories before reaching the end of the current plan, effectively maintaining safety through continuous replanning even without formal recursive feasibility guarantees.

IX. Simulation Results

We performed all the simulations on an `AlienWare Aurora R8` desktop computer with an Intel® Core ™ i9 CPU ×16, 64 GB of RAM. The operating system is Ubuntu 22.04 LTS, and we used ROS2 Humble for SANDO's implementation. Other methods were implemented in ROS1 Noetic/Melodic, so we dockerized them and ran them in ROS1 Noetic/Melodic on the same machine for benchmarking. Table II lists the SANDO system parameters that remain fixed across all simulations, and Table III summarizes the per-experiment configuration parameters for both simulation and hardware experiments (Section X). For all baseline methods, we used their default parameter values (planning horizon, number of trajectory pieces, etc.), with one exception: in the static forest benchmark, EGO-Swarm2's default planning horizon of 7.5 m caused frequent collisions even in the easy environment, so we increased it to 20.0 m to achieve a higher success rate (see Table VI).

A. Benchmarking in Standardized Static Environments

To compare SANDO against state-of-the-art methods, we first performed benchmarking experiments in a standardized static environment, as shown in Fig. 6. We evaluated 61 cases by varying the goal position from −3 to 3 m in 0.1 m increments along the y-axis. For fair comparison, we used the same start and goal positions, dynamic constraints, and safe flight corridor constraints. The dynamic constraints were set to $\boldsymbol{v}_{\max} = 1.0$ m/s, $\boldsymbol{a}_{\max} = 2.0$ m/s^2, and $\boldsymbol{j}_{\max} = 3.0$ m/s^3. We compared SANDO against FASTER [2] and SUPER [4], which are state-of-the-art static environment planners that both use safe flight corridors. Since this benchmark has no dynamic obstacles, SANDO uses spatial safe flight corridors

TABLE II. SANDO system parameters across all simulations.

Module	Symbol	Value	Parameter
Static Heat Map	α^s	5.0	Intensity scale
	p^s	2	Decay exponent
	R^s	$3.0 \cdot r_{\text{drone}}$	Halo radius
	$H_{\max}$	50.0	Maximum heat
Dynamic Heat Map	α_0^d	1.0	Base cost scale
	α_1^d	2.0	Tube penalty scale
	p^d, q^d	2	Decay exponents
	γ_k	$v_{\max}^{\text{obs}}$	Tube growth rate
	τ_{ratio}^d	0.5	Time weight ratio
Time Alloc.	Δf	0.1	Factor step size
	κ	0.4	Window half-width
Tracker	α	0.9	AEKF forgetting factor
Solver	N/A	Gurobi	MIQP solver [32]

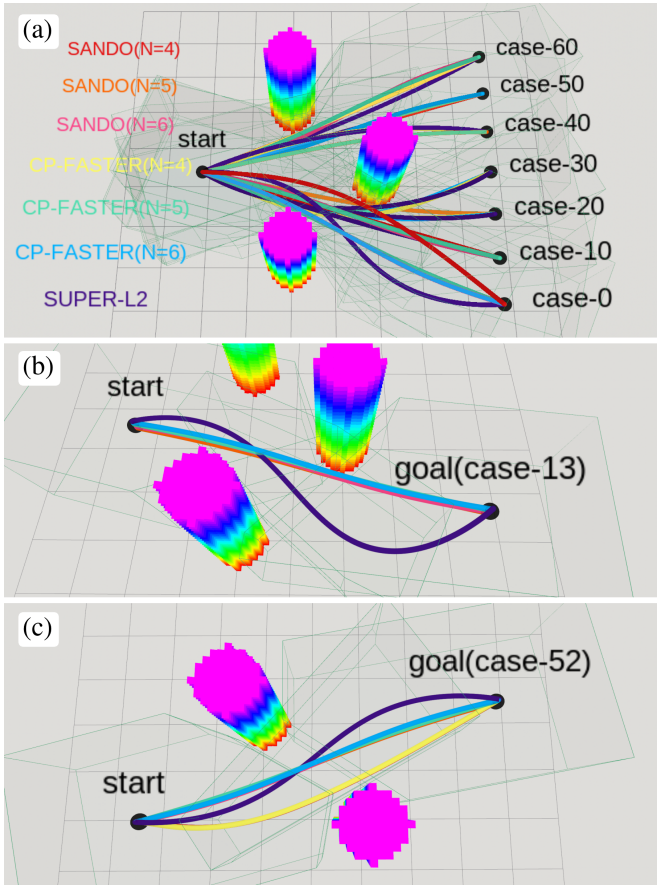


Fig. 6. Standardized Benchmarking: The environment used for benchmarking SANDO against state-of-the-art methods in static environments. Start position to the left and goal positions are the right. The safe flight corridors used as safe constraints shown as green polytopes. Trajectory color-coded according to the planner. (a) shows trajectories for 10 different cases with different start and goal positions. (b) and (c) show a close-up view of one of the cases. Instead of enforcing hard safe flight corridor constraints, SUPER uses soft constraints, where the trajectory is encouraged to go close to the center of overlapping polytopes, which results in longer trajectories as shown in (b) and (c).

(SSFCs) rather than STSFCs; an SSFC is a single-time-layer STSFC (i.e., $C[0][p]$ only), identical to FASTER's corridor generation, since obstacle positions do not change over time. FASTER uses MIQP-based hard constraints, while SUPER uses soft constraints. SANDO and FASTER use Gurobi [32] as the MIQP solver, while SUPER uses LBFGS [37] as the soft-constraint solver. For a fair comparison, we relaxed SUPER to use per-axis dynamic constraints (L_∞ norm), since SANDO and FASTER enforce per-axis constraints. FASTER only applies dynamic constraints at the very first control points of each piece, so we also report the results of FASTER with additional dynamic constraints at all control points, denoted as FASTER (CP). The original FASTER is denoted as FASTER (orig.). For SANDO and FASTER, we set the number of pieces from 4 to 6.

The metrics used in the table are defined as follows. $R_{\text{succ}}^{\text{opt}}$ [%] (optimization success rate; optimization completes without failure); $T_{\text{opt}}^{\text{per}}$ [ms] (per-optimization runtime; since SANDO and FASTER perform iterative time allocation and trajectory optimization, we report the average runtime of each individual optimization); $T_{\text{opt}}^{\text{total}}$ [ms] (total optimization runtime); T_{trav} [s] (trajectory travel time); L_{path} [m] (total path length); $S_{\text{jerk}} = \int_0^{T_{\text{trav}}} \|\mathbf{j}(t)\|\, dt$ [m/s^2] (L1 jerk integral, where $\mathbf{j}(t) = \dddot{\mathbf{p}}(t)$ is the trajectory jerk; smoothness); ρ_{sfc}, ρ_{vel}, ρ_{acc}, ρ_{jerk} [%] (SFC/velocity/acceleration/jerk constraint violation rates; the percentage of trajectory points that violate the respective constraints, with $\rho_{\text{acc/jerk}}$ denoting the combined acceleration and jerk violation rate when reported jointly). Unlike SANDO and FASTER, SUPER optimizes spatial trajectories combined with temporal allocation in a single optimization problem, so we report the per-optimization runtime as the total optimization runtime since it does not have iterative time allocation. FASTER and SUPER have two trajectory optimization approaches (exploratory and safe), but for this benchmarking we only report the exploratory trajectory optimization since it yields better performance. SUPER and SANDO use multi-threading for parallel optimization, while FASTER is single-threaded. We report both single-threaded and multi-threaded results for SANDO to show the benefit of multi-threading.

Table IV summarizes the benchmarking results. As N increases, both SANDO and FASTER achieve better trajectory performance (shorter travel time) at the cost of increased computation time, since trajectories with more segments have larger degrees of freedom. Note that FASTER (orig.) achieves fastest travel time at $N = 5$ but with a high velocity violation rate of 38.0 %, while FASTER (CP) achieves zero velocity violation since it applies dynamic constraints at all control points. Multi-threaded SANDO incurs a slightly higher per-optimization time $T_{\text{opt}}^{\text{per}}$ than single-threaded due to thread overhead (e.g., 6.8 vs. 6.1 ms at $N = 5$, 17.2 vs. 16.1 ms at $N = 6$), but achieves substantially faster total optimization time $T_{\text{opt}}^{\text{total}}$ due to parallel execution (e.g., 9.7 vs. 18.2 ms at $N = 5$, 19.7 vs. 28.0 ms at $N = 6$). No method exhibited acceleration or jerk constraint violations. SUPER reports SFC and velocity constraint violations because it uses

TABLE III. Per-experiment SANDO configuration. "N/A" indicates the parameter is not applicable.

Parameter	Simulation (Sec. IX)				Hardware (Sec. X)	
	Standardized (Sec. IX-A, IX-B)	Static Forest (Sec. IX-C)	Dynamic w/ GT (Sec. IX-D, IX-E)	Dynamic w/o GT (Sec. IX-F)	Static (Exp. 1–6)	Dynamic (Exp. 7–16)
Pieces N	4–6	5	5	5	5	5
Polytopes per layer P	3	3	3	2, 3	3	2
Drone radius r_{drone} [m]	0.1	0.1	0.1	0.1	0.45	0.45
Max obs. velocity $v_{\max}^{\text{obs}}$ [m/s]	N/A	N/A	0.5	0.5	N/A	0.25
A* heat weight w_{heat}	5.0	5.0	5.0	5.0	5.0	5.0 | 20.0
Estimation error ϵ [m]	N/A	N/A	0.0	0.0	N/A	0.0
Safety margin r_{margin} [m]	0.1	0.1	0.1	0.1	0.2	0.2
Max time factor $f_{\max}$	2.5	2.5	2.5	2.5	3.0	3.0
Unknown space inflation	N/A	on	on	on	off	off
Sensor	N/A	MID-360	N/A	D435	MID-360	MID-360

soft constraints; however, it achieves fastest computation time while performing spatiotemporal optimization. Compared to FASTER, SANDO achieves consistently faster computation across all N while maintaining the same trajectory performance and no constraint violations. Overall, SUPER achieves the fastest computation time due to its MINCO-based [21] spatiotemporal optimization, but at the cost of constraint violations. Among hard-constraint methods, SANDO and FASTER (CP) achieve the best trajectory performance with zero constraint violations, and SANDO is consistently computationally faster than FASTER across all N, especially with multi-threading. These results illustrate the fundamental trade-off between N (and similarly P) and computational cost: larger N and P increase the MIQP's degrees of freedom and the number of binary assignment variables ($N \times P$), improving trajectory quality but increasing solve time. In dynamic environments where fast replanning is critical, we use $N = 5$ and $P \in \{2, 3\}$ as a practical operating point that balances trajectory quality against real-time computation (see Table III).

B. Effectiveness of Variable Elimination

We also benchmarked SANDO with and without variable elimination (VE) (see Section VI-A.1) to evaluate the effectiveness of the technique. The benchmark was performed in the same standardized static environment with $N = 4, 5, 6$ with multi-threaded SANDO under the same dynamic constraints as in Section IX-A. Table V summarizes the results. The metrics used in the table are the same as those in Section IX-A except that we report all the constraint violation rates as a single value ρ_{viol} [%] (the maximum rate among SFC/velocity/acceleration/jerk constraint violations). VE achieves identical trajectory performance to the non-VE baseline across all N while reducing per-optimization time by up to 7.4×, confirming that, as expected, variable elimination reduces computation time while producing the same optimal solution since the underlying optimization problem is equivalent. This is because VE reduces the number of decision variables and constraints in the MIQP while effectively solving the original problem, so the same optimal solution is obtained with much faster computation.

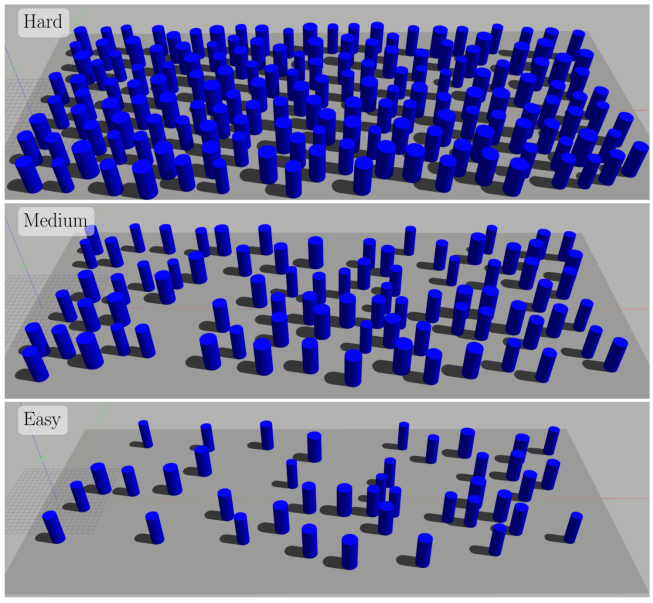


Fig. 7. Static Benchmarking: The static forest Gazebo environment used for benchmarking SANDO against state-of-the-art methods.

C. Benchmarking against State-of-the-Art Methods in Static Environments

To evaluate SANDO's full planning pipeline in realistic settings, we benchmarked it against state-of-the-art methods in obstacle-rich static forest environments at three difficulty levels. Static cylindrical obstacles (radius 1.0–1.5 m, height 6 m) were placed randomly, occupying a 100 m × 40 m area (Fig. 7). Three difficulty levels are defined by the fraction of the area occupied by obstacles: Easy (5%), Medium (10%), and Hard (20%). The agent starts at $(0, 0, 3)$ m and the goal is $(105, 0, 3)$ m.

We benchmarked SANDO against EGO-Swarm2 [17], SUPER [4], and FASTER [2]. To simulate LiDAR data, we used the `livox_ros_driver2` package, which provides a ROS 2 interface for the Livox MID-360 LiDAR sensor; all methods receive the same sensor data for fair comparison. The dynamic constraints were set to $\boldsymbol{v}_{\max} = 5.0$ m/s, $\boldsymbol{a}_{\max} = 20.0$ m/s^2, and $\boldsymbol{j}_{\max} = 100.0$ m/s^3. SANDO and

TABLE IV. Local trajectory optimization benchmarking results (computation time, performance, and constraint violation). SANDO and FASTER (CP) with $N = 6$ achieve the best trajectory performance (shortest travel time) while maintaining no constraint violations, but SANDO achieves substantially faster computation time than FASTER (CP) due to multi-threading. We mark in **green** the best value in each column and in **red** the worst value.

Algorithm		Thread	N	Success	Computation Time		Performance			Constraint Violation		
				$R_{\text{succ}}^{\text{opt}}$ [%]	$T_{\text{opt}}^{\text{per}}$ [ms]	$T_{\text{opt}}^{\text{total}}$ [ms]	T_{trav} [s]	L_{path} [m]	S_{jerk} [m/s^2]	ρ_{SFC}[%]	ρ_{vel}[%]	$\rho_{\text{acc/jerk}}$[%]
SUPER	(L_2)	multi	–	**100.0**	0.6		**14.1**	**6.8**	**1.3**	0.2	0.5	**0.0**
	(L_∞)			**100.0**	**0.5**		13.6	**6.9**	1.5	**0.3**	4.0	**0.0**
FASTER	(orig.)	single	4	**93.4**	7.9	48.1	13.0	**6.8**	1.4	**0.0**	5.0	**0.0**
	(CP)			**93.4**	4.5	36.5	13.2	**6.8**	1.4	**0.0**	**0.0**	**0.0**
SANDO		single		**93.4**	1.3	12.7	13.2	**6.8**	1.4	**0.0**	**0.0**	**0.0**
		multi		**93.4**	1.3	3.2	13.2	**6.8**	1.4	**0.0**	**0.0**	**0.0**
FASTER	(orig.)	single	5	**100.0**	16.8	31.3	**8.5**	**6.8**	**8.4**	**0.0**	**38.0**	**0.0**
	(CP)			**100.0**	18.0	67.6	11.1	**6.8**	1.9	**0.0**	**0.0**	**0.0**
SANDO		single		**100.0**	6.1	18.2	11.1	**6.8**	1.9	**0.0**	**0.0**	**0.0**
		multi		**100.0**	6.8	9.7	11.1	**6.8**	1.9	**0.0**	**0.0**	**0.0**
FASTER	(orig.)	single	6	**100.0**	**24.6**	**94.9**	9.8	**6.8**	2.8	**0.0**	8.2	**0.0**
	(CP)			**100.0**	20.2	56.8	9.8	**6.8**	2.8	**0.0**	**0.0**	**0.0**
SANDO		single		**100.0**	16.1	28.0	9.8	**6.8**	2.8	**0.0**	**0.0**	**0.0**
		multi		**100.0**	17.2	19.7	9.8	**6.8**	2.8	**0.0**	**0.0**	**0.0**

TABLE V. Variable elimination (VE) benchmarking results in standardized environment. VE achieves identical trajectory performance (travel time, path length, jerk, and constraint violations) to the non-VE baseline across all N, while reducing per-optimization time by up to 7.4× at $N = 4$, 1.9× at $N = 5$, and 1.4× at $N = 6$. We highlight the best and worst values for each N in **green** and **red**, respectively.

N	VE	$R_{\text{succ}}^{\text{opt}}$ [%]	$T_{\text{opt}}^{\text{per}}$ [ms]	T_{trav} [s]	L_{path} [m]	S_{jerk} [m/s^2]	ρ_{viol} [%]
4	**Yes**	93.4	**1.3**	13.2	6.8	1.4	0.0
	No	93.4	**9.6**	13.2	6.8	1.4	0.0
5	**Yes**	**100.0**	**6.8**	11.1	6.8	1.9	0.0
	No	**100.0**	**12.8**	11.1	6.8	1.9	0.0
6	**Yes**	**100.0**	**17.2**	9.8	6.8	2.8	0.0
	No	**100.0**	**24.7**	9.8	6.8	2.8	0.0

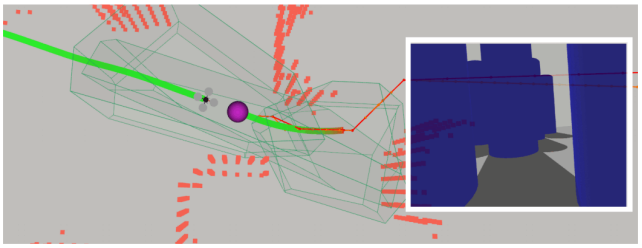


Fig. 8. Rviz Visualization of Static Benchmarking (Hard Environment): One of the simulation runs in the hard static environment. The purple dot is the replanning point (Point A), the green polytopes are the safe flight corridors, and the color-coded trajectory is the optimized trajectory, where green is faster and red is slower. The orange dots are occupied voxels, the orange line is the original global path, and the red line is a smoother version of the global path used for SFC generation.

FASTER enforce hard L_∞ dynamic constraints, while EGO-Swarm2 and SUPER use soft constraints. EGO-Swarm2 and SUPER originally use L_2 norms for dynamic constraints, but we relaxed them to L_∞ norms for fair comparison since SANDO and FASTER enforce per-axis constraints. We performed 10 simulations per difficulty level with the same evaluation metrics in Section IX-A with the addition of R_{succ} [%] (overall success rate; the percentage of runs in which the agent reaches the goal without collision) and T_{replan} [ms] (total replanning computation time). Fig. 8 shows a visualization of one of SANDO's simulation runs in RViz.

Table VI summarizes the results. SANDO achieves a 100% success rate across all difficulty levels, whereas FASTER drops to 90% and 70% in the medium and hard cases, respectively, and EGO-Swarm2 drops to 50–60% in the hard case. SANDO also achieves the fastest travel time and the lowest total optimization computation time $T_{\text{opt}}^{\text{total}}$ across all cases. Since SUPER and FASTER use both safe and exploratory trajectory optimization, we report the computation time of both trajectories in $T_{\text{opt}}^{\text{total}}$. SANDO maintains no constraint violations in velocity, acceleration, and jerk owing to its hard-constraint formulation. Although FASTER also enforces hard constraints, it exhibits velocity violations of 7.9% in the hard case, likely due to the constraints only being applied at the first control point of each piece. SUPER shows high jerk violation rates (8–12.7%) because its soft-constraint solver does not strictly enforce dynamic limits. Similarly, EGO-Swarm2 exhibits velocity violations (3–8.6%) due to its soft-constraint optimization. In terms of smoothness, SANDO's jerk integral S_{jerk} is lower than SUPER's and FASTER's, but higher than EGO-Swarm2's. Overall, SANDO achieves the best overall performance with the highest success rate, fastest travel time, and no constraint violations across all difficulty levels.

D. SANDO in Dynamic Environments

We next evaluated SANDO in environments containing both static and dynamic obstacles to test its spatiotemporal collision avoidance capability. The environment spans a 100 m × 40 m forest area populated with static cylindrical obstacles and dynamic obstacles modeled as 0.8 m cubes following trefoil knot trajectories with randomized parameters (position, scale, speed, and time offset) (Fig. 9). Three difficulty levels are defined: Easy (50 obstacles, ~33

TABLE VI. Benchmark results against state-of-the-art methods in static environments. SANDO outperforms the other methods in terms of travel time and achieves a 100% success rate. Since SUPER performs global path planning for both exploratory and safe trajectories, we list the corresponding computation times as Exploratory | Safe in the Global Path Planning Computation Time column. We mark in **green** the best value and in **red** the worst value in each column per case.

Env	Algorithm	Constr.		Success	Comp. Time		Performance			Constraint Violation		
				R_{succ} [%]	$T_{\text{opt}}^{\text{total}}$ [ms]	T_{replan} [ms]	T_{trav} [s]	L_{path} [m]	S_{jerk} [m/s^2]	ρ_{vel} [%]	ρ_{acc} [%]	ρ_{jerk} [%]
Easy	EGO-Swarm2	Soft	L_2	**100.0**	5.4	**6.2**	25.7	108.5	146.0	7.7	**0.0**	**0.0**
			L_∞	**100.0**	5.7	6.5	25.0	107.2	**142.4**	**8.6**	**0.0**	**0.0**
	SUPER	Soft	L_2	**100.0**	8.1 | 23.8	**32.0**	**26.0**	**122.2**	1291.0	**0.0**	**0.0**	**8.3**
			L_∞	**100.0**	8.4 | 23.4	31.9	25.7	121.1	**1305.4**	**0.0**	**0.0**	8.0
	FASTER	Hard	L_∞	**100.0**	**19.4** | 38.4	23.9	22.8	**105.5**	266.9	4.4	**0.0**	**0.0**
	SANDO	Hard	L_∞	**100.0**	**3.7**	12.3	**21.6**	105.6	184.9	**0.0**	**0.0**	**0.0**
Medium	EGO-Swarm2	Soft	L_2	**100.0**	5.4	6.0	**27.2**	109.4	196.7	**7.0**	**0.0**	**0.0**
			L_∞	**100.0**	4.7	**5.2**	**27.2**	109.0	186.7	6.2	**0.0**	**0.0**
	SUPER	Soft	L_2	**100.0**	8.5 | 22.5	**31.1**	25.8	**121.5**	1358.9	**0.0**	**0.0**	8.4
			L_∞	**100.0**	8.4 | 22.1	30.6	25.7	120.6	**1369.8**	**0.0**	**0.0**	**8.6**
	FASTER	Hard	L_∞	**90.0**	**22.2** | 38.4	26.7	23.1	**105.7**	320.8	4.5	**0.0**	**0.0**
	SANDO	Hard	L_∞	**100.0**	**4.5**	14.9	**21.6**	106.1	239.7	**0.0**	**0.0**	**0.0**
Hard	EGO-Swarm2	Soft	L_2	**50.0**	23.5	26.8	**34.2**	120.4	311.7	3.0	**0.0**	**0.0**
			L_∞	60.0	21.9	24.5	32.3	**121.6**	296.0	5.1	**0.0**	**0.0**
	SUPER	Soft	L_2	**100.0**	8.8 | 18.9	27.9	24.6	114.9	1504.2	**0.0**	**0.0**	11.9
			L_∞	**100.0**	9.2 | 19.5	28.8	24.8	115.4	**1527.3**	**0.0**	**0.0**	**12.7**
	FASTER	Hard	L_∞	70.0	**35.0** | 36.2	**41.4**	30.6	**107.9**	1348.7	**7.9**	**0.0**	**0.0**
	SANDO	Hard	L_∞	**100.0**	**6.1**	**21.6**	**22.2**	108.7	581.4	**0.0**	**0.0**	**0.0**

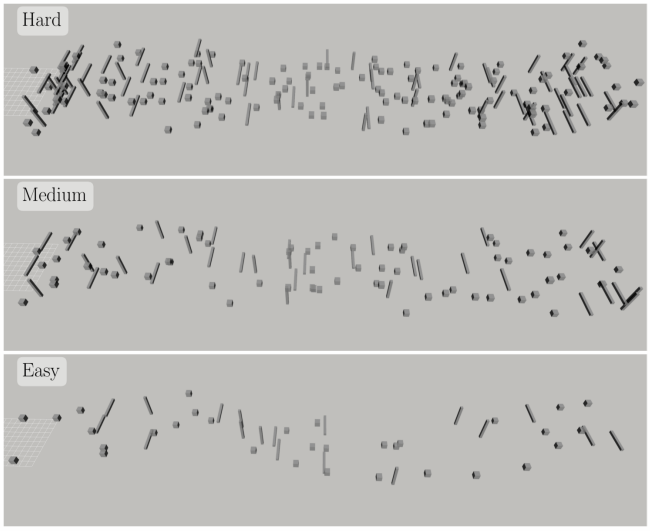


Fig. 9. Dynamic Benchmarking: The dynamic Gazebo environment used for benchmarking SANDO against state-of-the-art methods at three difficulty levels (Easy, Medium, and Hard). The environment contains a mix of static cylindrical obstacles and dynamic cube obstacles following trefoil knot trajectories with randomized parameters.

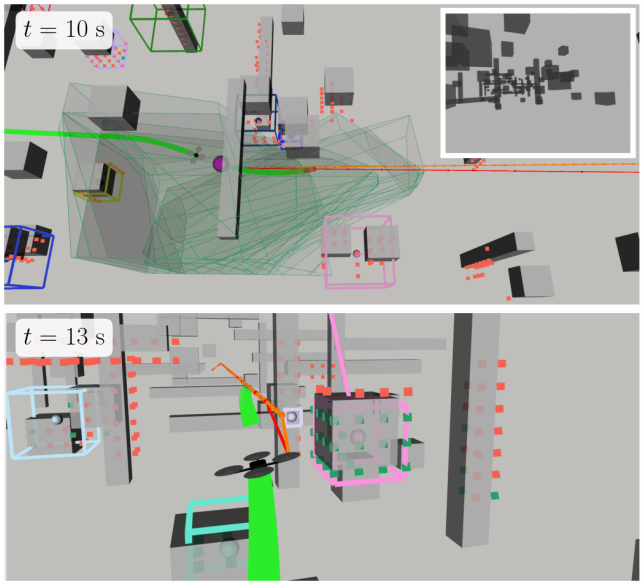


Fig. 10. Rviz Visualization of Dynamic Benchmarking (Hard Environment): We use the same visualization scheme as in Fig. 8. The estimated AABBs of the dynamic obstacles are shown as colored boxes and the predicted trajectories of the dynamic obstacles are shown as colored lines. The spatiotemporal safe flight corridors (STSFC) are shown as green polytopes. Notice that STSFCs have many layers in the time dimension, and hence many more polytopes than the static case.

dynamic), Medium (100 obstacles, ~65 dynamic), and Hard (200 obstacles, ~130 dynamic), with approximately 65% of obstacles being dynamic. The agent starts at $(0, 0, 2)$ m with a goal at $(105, 0, 2)$ m.

We benchmarked SANDO against EGO-Swarm2 [17], I-MPC [15], and FAPP [7] with 10 simulations per difficulty level. All methods use L_∞ dynamic constraints set to $v_{\max} = 5.0\,\text{m/s}$, $a_{\max} = 20.0\,\text{m/s}^2$, and $j_{\max} = 100.0\,\text{m/s}^3$. For I-MPC, we swept over six combinations of velocity and acceleration limits ($v_{\max} \in \{1, 2, 3, 4, 5\}\,\text{m/s}$ with matched $a_{\max}$, plus $v_{\max} = 5\,\text{m/s}$, $a_{\max} = 20\,\text{m/s}^2$) because at the nominal limits ($v_{\max} = 5\,\text{m/s}$, $a_{\max} = 20\,\text{m/s}^2$), I-MPC reports large constraint violations. Constraint violations for all methods are evaluated against the shared limits $v_{\max} = 5\,\text{m/s}$ and $a_{\max} = 20\,\text{m/s}^2$. Each method uses different inputs for dynamic obstacles. For instance, EGO-Swarm2 does not have a built-in dynamic obstacle estimator/predictor, but I-MPC and FAPP have built-in estimators/predictors. For a fair comparison, we feed the ground truth positions and future predictions of dynamic obstacles to EGO-Swarm2 since it only receives a quintic polynomial prediction of each dynamic obstacle with a clearance radius. For I-MPC, we feed the ground truth

positions, velocities, and sizes of dynamic obstacles since that is what their built-in `fake_detector` provides. For FAPP, we also feed the ground truth positions and velocities. FAPP uses a fixed squared-distance threshold of $1.96\,\mathrm{m}^2$ in its obstacle avoidance cost, corresponding to an effective avoidance radius of $\sqrt{1.96} \approx 1.4\,\mathrm{m}$ from the obstacle center regardless of the actual obstacle size; i.e., it does not adapt per obstacle. In contrast, for this benchmarking, SANDO receives only the current position of dynamic obstacles, and hence SANDO receives the least information about dynamic obstacles compared to other methods.

Table VII summarizes the results, and Fig. 10 shows a visualization of one of SANDO's simulation runs in RViz. SANDO achieves a 100% success rate across all difficulty levels, while all other methods exhibit failures in one or more cases. SANDO (P=3) also achieves the shortest path length in all cases and the fastest travel time in easy and hard. Most notably, SANDO maintains zero constraint violations in velocity, acceleration, and jerk across all cases, demonstrating that hard constraints in the MIQP formulation reliably enforce dynamic feasibility even in dense dynamic environments. EGO-Swarm2 achieves 100% success in easy and medium but drops to 80% in hard, with velocity violations of 5.1–11.8% across all cases. FAPP achieves 80% success in easy and medium and 50% in hard, with small velocity violations (3.1–8.1%) and no acceleration or jerk violations. Among I-MPC variants, lower velocity and acceleration limits reduce constraint violations, while higher limits lead to constraint violations without improved success rates. I-MPC's jerk violations are reported as "-" because it does not enforce jerk constraints. These results show that SANDO's STSFC approach with worst-case reachable set inflation maintains safety under the most conservative obstacle information assumption.

E. STSFC Ablation

To evaluate the effectiveness of the STSFC approach, we compared it against a worst-case baseline that inflates all dynamic obstacles by the maximum time horizon. The STSFC approach inflates obstacles per layer using $r_n = v_{\max}^{\text{obs}} \cdot (n{+}1) \cdot dt + \epsilon$, where $(n{+}1) \cdot dt$ is the end time of layer n (Section IV), whereas the worst-case baseline uses $r = v_{\max}^{\text{obs}} \cdot T_{\text{traj}} + \epsilon$ for all layers, where $T_{\text{traj}} = N \cdot dt$ is the total trajectory duration. We conducted the comparison in the Hard dynamic environment from Section IX-D at two maximum velocities ($v_{\max}$ = 2.5 and 5.0 m/s) to examine how corridor inflation interacts with agent speed.

Table VIII summarizes the results. At $v_{\max} = 2.5\,\mathrm{m/s}$, the worst-case baseline achieves only 80% success rate because the large uniform inflation radius results in replanning failures and stoppage when the agent gets hit by dynamic obstacles. The STSFC approach maintains a 100% success rate by inflating obstacles proportionally to each time layer, preserving more free space in earlier layers while still guaranteeing safety. STSFC also achieves substantially lower computation time (8.9 vs. 15.5 ms), faster travel time (43.8 vs. 58.8 s), and a much smoother trajectory (jerk integral 330.6 vs. 1392.2 $\mathrm{m/s^2}$). At $v_{\max} = 5.0\,\mathrm{m/s}$, both approaches achieve 100% success, as the faster trajectory (and hence shorter traversal time) reduces the worst-case inflation radius and allows the worst-case approach to succeed without many replanning failures. Nevertheless, STSFC still provides lower computation time (5.0 vs. 6.4 ms), faster travel time (22.7 vs. 23.5 s), and a smoother trajectory (jerk integral 669.6 vs. 753.3 $\mathrm{m/s^2}$) than the worst-case baseline.

F. Dynamic Environments without Ground Truth Obstacle Knowledge

The dynamic benchmarks in Sections IX-D and IX-E provide all methods with ground truth obstacle information for fair comparison. In practice, however, this information is not available. This section evaluates SANDO in the same dynamic environments but without any ground truth obstacle knowledge, testing the full perception-to-planning pipeline.

The environment, obstacle configuration, dynamic constraints, start/goal positions, and trial settings are identical to those in Section IX-D. For sensing, we used a simulated Intel RealSense D435 depth camera (via the `realsense-ros` package) rather than the Livox MID-360 LiDAR used in the static benchmarks, because the simulated LiDAR produces too few points on the small dynamic obstacles for reliable detection. This is a simulation artifact; in hardware experiments (Section X), the real LiDAR sensor provides sufficient point density for dynamic obstacle detection. Point clouds are processed by the temporal occupancy grid and AEKF-based dynamic obstacle tracker (Section VII); the planner receives only estimated obstacle positions, bounding boxes, and predicted trajectories rather than ground truth. We evaluated two spatial polytope configurations ($P = 2$ and $P = 3$).

Table IX summarizes the results, where T_{STSFC} denotes the spatiotemporal safe flight corridor generation time. $P = 2$ generally achieves faster per-optimization time (3.0–4.6 ms vs. 4.6–8.5 ms for $P = 3$), while $P = 3$ produces smoother trajectories with lower jerk integrals. In the easy case, both configurations achieve high success rates (93% for $P = 2$, 95% for $P = 3$). In the medium case, $P = 2$ achieves 90% while $P = 3$ drops to 86%. In the hard case, success rates are 64% ($P = 2$) and 51% ($P = 3$). This is likely because a shorter replanning horizon and faster computation time with $P = 2$ allows more frequent replanning and quicker reactions to unexpected obstacles, which is beneficial in dense dynamic environments where perception uncertainty is high. Neither configuration produced any constraint violations across all difficulty levels.

Compared to the ground-truth results in Section IX-D, where SANDO achieves 100% success across all difficulties, the perception-only gap is small in the easy case (93–95% vs. 100%) but becomes larger in medium (86–90% vs. 100%) and hard (51–64% vs. 100%) cases. The primary failure mode is late detection of dynamic obstacles: when obstacles are not detected until they are close to the agent, the planner has insufficient time to generate a collision-free trajectory, resulting in collisions. This highlights the challenge

TABLE VII. Dynamic obstacle benchmarking results. We report success rate, computation time, flight performance, smoothness, and constraint violation metrics. We mark in **green** the best value and in **red** the worst value in each column per case.

Env	Algorithm	Success	Comp. Time	Performance			Constraint Violation		
		R_{succ} [%]	$T_{\text{opt}}^{\text{per}}$ [ms]	T_{trav} [s]	L_{path} [m]	S_{jerk} [m/s^2]	ρ_{vel} [%]	ρ_{acc} [%]	ρ_{jerk} [%]
Easy	EGO-Swarm2	**100.0**	5.7	25.4	107.5	**130.4**	11.8	**0.0**	**0.0**
	I-MPC (v=1, a=1)	80.0	12.1	**97.3**	117.3	1029.6	**0.0**	**0.0**	-
	I-MPC (v=2, a=2)	90.0	12.3	52.7	115.6	1167.1	**0.0**	**0.0**	-
	I-MPC (v=3, a=3)	90.0	12.1	38.3	117.7	1254.6	**0.0**	**0.0**	-
	I-MPC (v=4, a=4)	90.0	**12.5**	29.6	104.8	1236.6	1.3	**0.0**	-
	I-MPC (v=5, a=5)	**70.0**	10.9	25.7	118.8	1322.9	**56.2**	**0.0**	-
	I-MPC (v=5, a=20)	80.0	11.9	26.2	**131.1**	**2383.3**	47.2	**4.8**	-
	FAPP	80.0	**0.6**	22.8	106.3	178.9	8.1	**0.0**	**0.0**
	SANDO (P=2)	**100.0**	2.5	24.9	105.8	559.1	**0.0**	**0.0**	**0.0**
	SANDO (P=3)	**100.0**	3.0	**21.8**	**105.5**	236.0	**0.0**	**0.0**	**0.0**
Medium	EGO-Swarm2	**100.0**	8.5	26.2	108.3	**145.3**	10.4	**0.0**	**0.0**
	I-MPC (v=1, a=1)	70.0	22.7	**96.7**	123.6	1037.0	**0.0**	**0.0**	-
	I-MPC (v=2, a=2)	70.0	**22.9**	53.3	119.9	1181.9	**0.0**	**0.0**	-
	I-MPC (v=3, a=3)	60.0	22.2	38.4	122.2	1123.2	**0.0**	**0.0**	-
	I-MPC (v=4, a=4)	**30.0**	20.2	30.1	120.9	1138.6	3.5	**0.0**	-
	I-MPC (v=5, a=5)	50.0	17.6	25.1	120.8	1100.2	**60.3**	**0.0**	-
	I-MPC (v=5, a=20)	**30.0**	17.3	27.0	**134.0**	**2585.8**	56.9	**4.2**	-
	FAPP	80.0	**0.7**	**21.7**	106.7	240.0	4.8	**0.0**	**0.0**
	SANDO (P=2)	**100.0**	2.9	24.6	106.4	641.6	**0.0**	**0.0**	**0.0**
	SANDO (P=3)	**100.0**	3.8	21.8	**105.9**	356.3	**0.0**	**0.0**	**0.0**
Hard	EGO-Swarm2	80.0	18.7	29.6	114.2	**221.2**	5.1	**0.0**	**0.0**
	I-MPC (v=1, a=1)	40.0	31.7	**85.1**	121.3	940.1	**0.0**	**0.0**	-
	I-MPC (v=2, a=2)	30.0	30.9	51.0	123.3	942.8	**0.0**	**0.0**	-
	I-MPC (v=3, a=3)	30.0	**32.5**	36.7	119.9	1047.0	**0.0**	**0.0**	-
	I-MPC (v=4, a=4)	**0.0**	-	-	-	-	-	-	-
	I-MPC (v=5, a=5)	10.0	26.8	28.5	132.3	983.9	**44.6**	**0.0**	-
	I-MPC (v=5, a=20)	10.0	30.0	26.4	**198.0**	**3688.7**	41.7	**36.3**	-
	FAPP	50.0	**0.6**	27.9	111.3	587.1	3.1	**0.0**	**0.0**
	SANDO (P=2)	**100.0**	3.6	27.0	110.0	1198.4	**0.0**	**0.0**	**0.0**
	SANDO (P=3)	**100.0**	5.0	**22.7**	**108.5**	669.6	**0.0**	**0.0**	**0.0**

TABLE VIII. SFC ablation study. We compare Worst-Case SFC and Spatiotemporal SFC (STSFC) at different maximum velocities in the Hard dynamic environment. Neither approach exhibited any constraint violations. We highlight the **best** and **worst** value for each velocity.

$v_{\max}$ [m/s]	SFC Mode	R_{succ} [%]	$T_{\text{opt}}^{\text{per}}$ [ms]	T_{trav} [s]	L_{path} [m]	S_{jerk} [m/s^2]
2.5	Worst-Case	**80.0**	**15.5**	**58.8**	**111.1**	**1392.2**
	SANDO (STSFC)	**100.0**	**8.9**	**43.8**	**109.7**	**330.6**
5.0	Worst-Case	**100.0**	**6.4**	**23.5**	**108.2**	**753.3**
	SANDO (STSFC)	**100.0**	**5.0**	**22.7**	**108.5**	**669.6**

of perception uncertainty in dense dynamic environments. Nevertheless, these results confirm that SANDO can maintain a high success rate even without any ground truth obstacle information, showcasing the robustness of its full perception-to-planning pipeline in dynamic environments.

X. Hardware Experiments

To evaluate the performance of SANDO, we conducted hardware experiments in static and dynamic environments. Fig. 11 shows the UAV platform used in our experiments. For perception, we use a Livox Mid-360 LiDAR sensor, and for localization, we use onboard DLIO [38]. SANDO runs on an Intel™ NUC 13 with an Intel® Core ™ i7 CPU ×16, 64 GB of RAM, and for low-level control, we use PX4 [39] on a Pixhawk flight controller [40]. All perception, planning, control, and localization modules run onboard in real time, enabling fully autonomous operations. As summarized in Table III, we use $w_{\text{heat}} = 5.0$ for Experiments 7–10 (single obstacle) and increase it to $w_{\text{heat}} = 20.0$ for Experiments 11–16 (five obstacles) to account for the higher collision risk in denser environments. We also disabled unknown space inflation for the hardware experiments because the MID-360's field of view has many blind spots, and inflating unknown space results in excessive conservatism close to the UAV and prevents it from finding any feasible trajectory. As noted in Section VIII, with unknown space inflation turned off, the formal safety guarantee of Theorem 1 covers only tracked dynamic obstacles; untracked obstacles emerging from unobserved space are not accounted for in the corridor generation. Nevertheless, SANDO maintains practical safety through the combined effect of conservative obstacle inflation ($r_{\text{margin}} = 0.2\,\text{m}$, $r_{\text{drone}} = 0.45\,\text{m}$), continuous replanning, and heat map-based steering away from occluded areas, as

TABLE IX. Benchmark results in unknown dynamic environments. SANDO navigates using only pointcloud sensing (no ground truth obstacle trajectories). We highlight the **best** and **worst** value for each environment.

Env	P	Success	Comp. Time			Performance			Constr. Viol.
		R_{succ} [%]	$T_{\text{opt}}^{\text{per}}$ [ms]	T_{replan} [ms]	T_{STSFC} [ms]	T_{trav} [s]	L_{path} [m]	S_{jerk} [m/s^2]	ρ_{viol} [%]
Easy	2	**93.0**	**3.0**	**17.2**	**6.3**	**23.1**	**105.7**	**306.7**	**0.0**
	3	**95.0**	**4.6**	**19.9**	**7.2**	**22.6**	**105.6**	**217.9**	**0.0**
Medium	2	**90.0**	**3.7**	**22.1**	**8.5**	**24.0**	**106.6**	**472.3**	**0.0**
	3	**86.0**	**6.0**	**26.4**	**10.1**	**22.8**	**106.4**	**334.7**	**0.0**
Hard	2	**64.0**	**4.6**	**30.6**	**12.2**	**24.9**	**109.2**	**836.8**	**0.0**
	3	**51.0**	**8.5**	**37.4**	**14.7**	**23.7**	**108.1**	**576.0**	**0.0**



Fig. 11. Holybro PX4 Development Kit X500 is equipped with a Pixhawk flight controller and protective propeller guards. A Livox Mid-360 LiDAR is mounted on top for perception and localization. All modules run fully onboard: perception, planning, and localization on an Intel™ NUC 13, and low-level control on the Pixhawk flight controller.

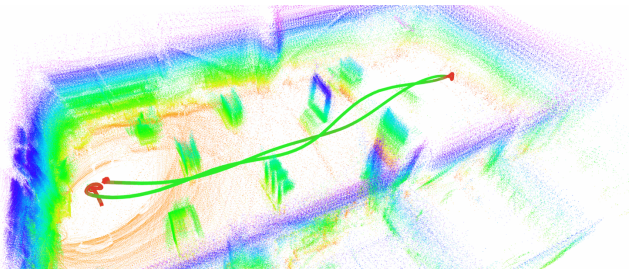


Fig. 12. Static environment experiment (Experiment 3): the UAV successfully navigates through a cluttered environment with static obstacles. The agent's speed is color-coded (green for faster, red for slower), and the point cloud is colored by height.

demonstrated by 16 collision-free hardware flights across all experiments.

A. Static Environments

We first evaluated SANDO in a static indoor environment where obstacles were placed in a 20 m × 8 m area. The UAV was tasked with flying from $x = 0.0$ m to $x = 20.0$ m and returning to the start. To assess performance across a range of speeds, we conducted six flights with velocity limits $v_{\max} \in \{1.0, 2.0, 3.0, 4.0, 5.0, 6.0\}$ m/s. The acceleration and jerk limits were set to $a_{\max} = 5.0\,\text{m/s}^2$ and $j_{\max} = 10.0\,\text{m/s}^3$ for the first four flights. For Experiments 5 and 6, $j_{\max}$ was reduced to $7.5\,\text{m/s}^3$ to mitigate the larger tracking errors observed at higher speeds. Fig. 12 shows the resulting trajectory overlaid on the LiDAR point cloud; the UAV successfully avoids all static obstacles across the entire speed range.

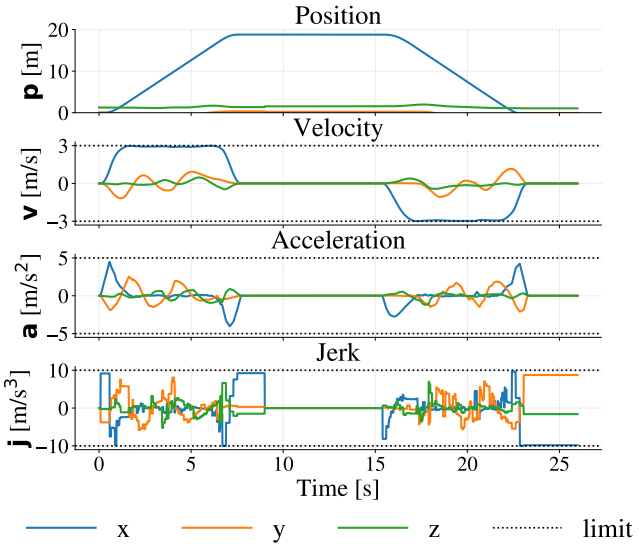


Fig. 13. UAV's velocity profile in Experiment 3. UAV satisfies the dynamic constraints and the velocity profile is smooth. When the UAV reaches the goal, it rotates in place to face the starting point, which causes the velocity, acceleration, and jerk to drop to zero in the middle of the flight.

TABLE X. Hardware flight computation times in static environments with increasing velocity limits. All use $a_{\max} = 5$ m/s^2 and $j_{\max} = 10$ m/s^3.

Exp.	$v_{\max}$ [m/s]	Computation Time [ms]			
		T_{replan}	T_{global}	T_{SSFC}	T_{opt}
1	1.0	35.8 ± 8.8	0.1 ± 0.1	6.1 ± 3.0	14.0 ± 6.3
2	2.0	35.7 ± 8.8	0.1 ± 0.1	6.5 ± 3.2	12.3 ± 6.4
3	3.0	34.5 ± 7.6	0.1 ± 0.1	7.7 ± 3.2	9.7 ± 5.2
4	4.0	34.8 ± 8.0	0.1 ± 0.1	8.9 ± 4.0	8.6 ± 4.2
5	5.0	33.9 ± 7.0	0.1 ± 0.1	7.9 ± 3.6	8.3 ± 4.0
6	6.0	32.6 ± 6.8	0.1 ± 0.1	7.7 ± 3.6	8.2 ± 4.3

Table X reports the computation times for all six flights, where T_{replan}, T_{global}, T_{SSFC}, and T_{opt} denote the average total replanning, global planning, spatial safe flight corridor (SSFC) generation, and trajectory optimization times, respectively. As in the static simulation benchmark (Section IX-A), SANDO uses SSFCs rather than STSFCs since there are no dynamic obstacles. Although the computation times are higher than in simulation due to the less powerful onboard computer, SANDO consistently maintains real-time performance, with average replanning times around 35 ms across

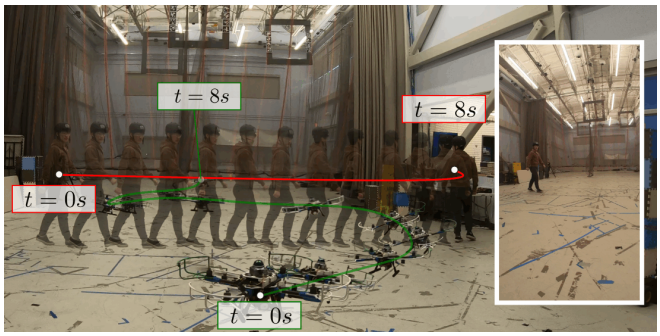


Fig. 14. Experiment 10: the UAV avoids a single dynamic obstacle moving at 0.5 m/s in an 8 m × 8 m area. Screenshots are taken every 0.5 s. The trajectory is color-coded by speed, and the onboard camera view is overlaid

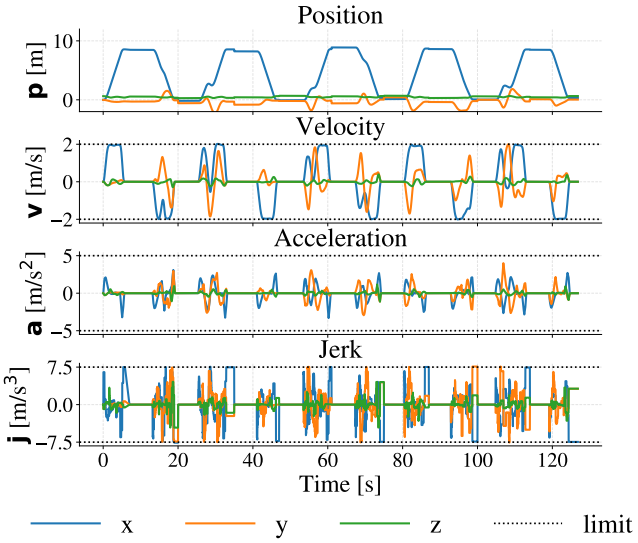


Fig. 15. UAV's velocity profile in Experiment 10. The UAV satisfies the dynamic constraints while flying back and forth around a dynamic obstacle.

TABLE XI. Hardware flight computation times in dynamic environments. All flights use $a_{\max} = 5$ m/s^2 and $j_{\max} = 10$ m/s^3.

Exp.	Obst. Type	Obst. Traj.	$v_{\max}$ [m/s]	Computation Time [ms]			
				T_{replan}	T_{global}	T_{STSFC}	T_{opt}
7	1 Dyn.	Line	2.0	21.4 ± 5.0	0.1 ± 0.0	4.9 ± 3.2	6.0 ± 2.4
8		Circle		22.2 ± 5.5	0.1 ± 0.1	5.0 ± 3.3	6.7 ± 2.9
9		Fig. Eight		22.1 ± 5.3	0.1 ± 0.0	4.9 ± 3.2	6.4 ± 2.8
10		Person		22.1 ± 6.0	0.1 ± 0.1	5.0 ± 3.2	6.4 ± 3.2
11	5 Dyn.	Line	2.0	24.6 ± 6.6	0.5 ± 0.5	6.2 ± 3.8	6.7 ± 3.3
12				20.3 ± 5.2	0.1 ± 0.1	5.0 ± 3.4	5.5 ± 2.1
13				21.3 ± 5.5	0.2 ± 0.3	5.0 ± 3.3	5.9 ± 2.7
14	5 Dyn. & Static	Line	2.0	21.9 ± 5.7	0.2 ± 0.3	5.0 ± 3.3	6.6 ± 3.1
15			3.0	21.4 ± 5.7	0.1 ± 0.1	5.1 ± 3.4	6.1 ± 2.8
16			4.0	21.2 ± 5.3	0.1 ± 0.2	4.9 ± 3.0	6.7 ± 2.8

all velocity limits. Fig. 13 shows the velocity profile for Experiment 3, confirming that the UAV satisfies the dynamic constraints while maintaining smooth velocity transitions throughout the flight.

B. UAV in Dynamic Environments

To evaluate SANDO in dynamic environments, we conducted hardware experiments involving one or five dynamic obstacles, each created by attaching an approximately 2.0 m-tall foam rectangular box to a wheeled platform. In Experiments 7 to 10, the UAV flew back and forth ($x = 0.0$ m to $x = 8.0$ m, 5 rounds) in an 8 m × 8 m area with a

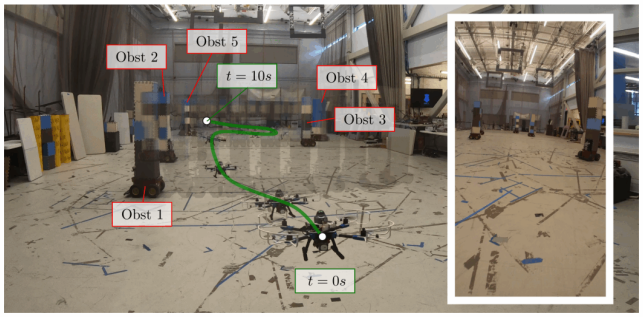
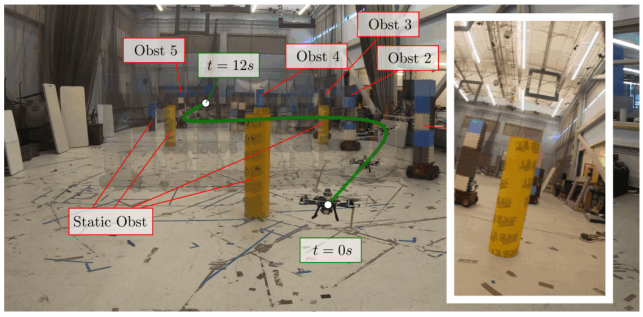


Fig. 16. Experiment 13: the UAV navigates among five dynamic obstacles moving at 0.5 m/s with random linear trajectories in an 8 m × 20 m area. Screenshots are taken every 1 s.



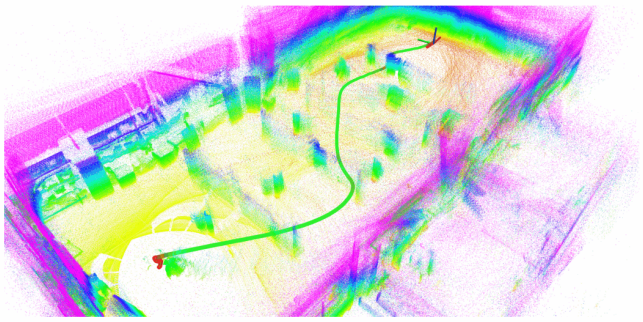


Fig. 17. Experiment 14: the UAV flies at $v_{\max} = 2.0$ m/s through an 8 m × 20 m area with five dynamic obstacles and additional static obstacles. Screenshots are taken every 1 s.

single dynamic obstacle moving at approximately 0.5 m/s: Experiments 7–9 used linear, circular, and figure-eight obstacle trajectories, while Experiment 10 (Fig. 14) used a person walking randomly to test SANDO's ability to handle unpredictable human motion. Experiments 11 to 13 (Fig. 16) had five dynamic obstacles in an 8 m × 20 m area, where each obstacle follows a linear trajectory at 0.5 m/s with random initial positions and directions. The agent was tasked with flying from $x = 0.0$ m to $x = 20.0$ m and returning to the start, similar to the static environment experiments. In Experiments 14 to 16 (Fig. 17), the same five dynamic obstacles were placed in an environment with additional static obstacles, and the UAV was tasked with flying at higher speeds ($v_{\max} \in \{2.0, 3.0, 4.0\}$ m/s) to further challenge SANDO's performance. The agent was tasked with flying from $x = 0.0$ m to $x = 20.0$ m, while avoiding both static and dynamic obstacles. The dynamic constraints were set to $v_{\max} = 2.0$ m/s, $a_{\max} = 5.0$ m/s^2, and $j_{\max} = 10.0$ m/s^3 for Experiments 7 to 13, and $v_{\max} \in \{2.0, 3.0, 4.0\}$ m/s,

$a_{\max} = 5.0\,\mathrm{m/s^2}$, and $j_{\max} = 10.0\,\mathrm{m/s^3}$ for Experiments 14 to 16. Fig. 15 shows the velocity profile for Experiment 10, confirming that the UAV satisfies the dynamic constraints throughout the flight.

Table XI summarizes the computation times for all ten dynamic environment experiments, where T_{STSFC} denotes the spatiotemporal safe flight corridor generation time. For the dynamic experiments, we used $P = 2$ spatial polytopes per time layer, whereas the static experiments used $P = 3$. This difference is because, as shown in Table IX, $P = 2$ achieves higher success rates than $P = 3$ in harder dynamic environments (64% vs. 51% in the hard case), since fewer polytopes per layer reduce the MIQP complexity and allow faster replanning, which is critical when the obstacles move. This also explains why the average replanning times in dynamic experiments (around 22 ms) are lower than in static environments. Overall, SANDO successfully avoids all dynamic obstacles across all ten experiments while maintaining real-time performance.

XI. Conclusions

This paper presented SANDO, a safe trajectory planner for 3D dynamic unknown environments. SANDO combines a heat map-based global planner with STSFC generation that inflates dynamic obstacles by worst-case reachable sets, a variable elimination technique that reduces hard-constraint MIQP optimization to as few as one free variable per axis (for $N = 4$), and an AEKF-based dynamic obstacle tracker. Simulations across standardized static benchmarks, obstacle-rich forests, and dynamic environments showed that SANDO consistently achieves the highest success rate with no constraint violations across all difficulty levels, and perception-only experiments without ground truth obstacle information confirmed robust performance under realistic sensing conditions. Hardware experiments on a quadrotor validated the approach.

The current framework has two main limitations. First, as discussed in Section VIII-E, SANDO does not guarantee recursive feasibility due to its moving subgoal structure. Second, the worst-case reachable set inflation can become overly conservative in dense environments, as evidenced by the perception-only results in Section IX-F.

XII. Acknowledgements

The authors would like to thank Lili Sun for her insightful comments, drone hardware design, and help with the hardware experiments; Lucas Jia for his assistance with the hardware experiments; Juan Rached for his help with the drone design and build; and Mason B. Peterson for his help with the ground robot setup.

References

[1] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, "Ego-planner: An esdf-free gradient-based local planner for quadrotors," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 478–485, 2021.

[2] J. Tordesillas, B. T. Lopez, M. Everett, and J. P. How, "Faster: Fast and safe trajectory planner for navigation in unknown environments," *IEEE Transactions on Robotics*, vol. 38, no. 2, pp. 922–938, 2022.

[3] C. Toumieh and D. Floreano, "High-speed motion planning for aerial swarms in unknown and cluttered environments," *IEEE Transactions on Robotics*, vol. 40, pp. 3642–3656, 2024.

[4] Y. Ren, F. Zhu, G. Lu, Y. Cai, L. Yin, F. Kong, J. Lin, N. Chen, and F. Zhang, "Safety-assured high-speed navigation for mavs," *Science Robotics*, vol. 10, no. 98, p. eado6187, 2025.

[5] J. Feng, J. Zhang, G. Zhang, S. Xie, Y. Ding, and Z. Liu, "Uav dynamic path planning based on obstacle position prediction in an unknown environment," *IEEE Access*, vol. 9, pp. 154679–154691, 2021.

[6] Z. Xu, Y. Xiu, X. Zhan, B. Chen, and K. Shimada, "Vision-aided uav navigation and dynamic obstacle avoidance using gradient-based b-spline trajectory optimization," *arXiv preprint arXiv:2209.07003*, 2022.

[7] M. Lu, X. Fan, H. Chen, and P. Lu, "Fapp: Fast and adaptive perception and planning for uavs in dynamic cluttered environments," *IEEE Transactions on Robotics*, vol. 41, pp. 871–886, 2025.

[8] Z. Zong, D. Li, X. Dong, Y. Cui, B. Yang, J. Xiang, and Z. Tu, "Risk-aware enabled path planning for drones flight in unknown environment," *Journal of Intelligent & Robotic Systems*, vol. 111, 2025.

[9] X. Fan, M. Lu, B. Xu, and P. Lu, "Flying in highly dynamic environments with end-to-end learning approach," *IEEE Robotics and Automation Letters*, vol. 10, no. 4, pp. 3851–3858, 2025.

[10] J. Lin, H. Zhu, and J. Alonso-Mora, "Robust vision-based obstacle avoidance for micro aerial vehicles in dynamic environments," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2682–2688, IEEE, 2020.

[11] R. Firoozi, A. Mir, G. S. Camps, and M. Schwager, "Oa-mpc: Occlusion-aware mpc for guaranteed safe robot navigation with unseen dynamic obstacles," *IEEE Transactions on Control Systems Technology*, vol. 33, no. 3, pp. 940–951, 2025.

[12] T. Liu, F. Zhang, F. Gao, and J. Pan, "Tight collision probability for uav motion planning in uncertain environment," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 1055–1062, 2023.

[13] C. Stamouli, L. Lindemann, and G. Pappas, "Recursively feasible shrinking-horizon mpc in dynamic environments with conformal prediction guarantees," in *6th Annual Learning for Dynamics & Control Conference*, pp. 1330–1342, PMLR, 2024.

[14] F. Quan, Y. Shen, P. Liu, X. Lyu, and H. Chen, "A state-time space approach for local trajectory replanning of an mav in dynamic indoor environments," *IEEE Robotics and Automation Letters*, vol. 10, no. 4, pp. 3438–3445, 2025.

[15] Z. Xu, H. Jin, X. Han, H. Shen, and K. Shimada, "Intent prediction-driven model predictive control for uav planning and navigation in dynamic environments," *IEEE Robotics and Automation Letters*, vol. 10, no. 5, pp. 4946–4953, 2025.

[16] J. Tordesillas and J. P. How, "Panther: Perception-aware trajectory planner in dynamic environments," *IEEE Access*, vol. 10, pp. 22662–22677, 2022.

[17] X. Zhou, X. Wen, Z. Wang, Y. Gao, H. Li, Q. Wang, T. Yang, H. Lu, Y. Cao, C. Xu, and F. Gao, "Swarm of micro flying robots in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm5954, 2022.

[18] Z. Xu, C. Suzuki, X. Zhan, and K. Shimada, "Heuristic-based incremental probabilistic roadmap for efficient uav exploration in dynamic environments," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 11832–11838, 2024.

[19] R. Deits and R. Tedrake, "Efficient mixed-integer planning for uavs in cluttered environments," in *2015 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 42–49, 2015.

[20] B. Landry, R. Deits, P. R. Florence, and R. Tedrake, "Aggressive quadrotor flight through cluttered environments using mixed integer programming," in *2016 IEEE international conference on robotics and automation (ICRA)*, pp. 1469–1475, IEEE, 2016.

[21] Z. Wang, X. Zhou, C. Xu, and F. Gao, "Geometrically constrained trajectory optimization for multicopters," *IEEE Transactions on Robotics*, vol. 38, no. 5, pp. 3259–3278, 2022.

[22] P. Fiorini and Z. Shiller, "Motion planning in dynamic environments using velocity obstacles," *The international journal of robotics research*, vol. 17, no. 7, pp. 760–772, 1998.

[23] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *2019 18th European control conference (ECC)*, pp. 3420–3431, Ieee, 2019.

[24] M. Chen and C. J. Tomlin, "Hamilton–jacobi reachability: Some recent theoretical advances and applications in unmanned airspace

management," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 1, no. 1, pp. 333–358, 2018.
[25] S. Liu, M. Watterson, K. Mohta, K. Sun, S. Bhattacharya, C. J. Taylor, and V. Kumar, "Planning dynamically feasible trajectories for quadrotors using safe flight corridors in 3-d complex environments," *IEEE Robotics and Automation Letters*, vol. 2, no. 3, pp. 1688–1695, 2017.
[26] R. Deits and R. Tedrake, "Computing large convex regions of obstacle-free space through semidefinite programming," in *Algorithmic Foundations of Robotics XI: Selected Contributions of the Eleventh International Workshop on the Algorithmic Foundations of Robotics*, pp. 109–124, Springer, 2015.
[27] D. Mellinger, A. Kushleyev, and V. Kumar, "Mixed-integer quadratic program trajectory generation for heterogeneous quadrotor teams," in *2012 IEEE international conference on robotics and automation*, pp. 477–483, IEEE, 2012.
[28] J. Nocedal and S. J. Wright, *Numerical optimization*. Springer, 2006.
[29] C. Richter, A. Bry, and N. Roy, "Polynomial trajectory planning for aggressive quadrotor flight in dense indoor environments," in *Robotics Research: The 16th International Symposium ISRR*, pp. 649–666, Springer, 2016.
[30] B. Zhou, J. Pan, F. Gao, and S. Shen, "Raptor: Robust and perception-aware trajectory replanning for quadrotor fast flight," *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1992–2009, 2021.
[31] G. E. Farin, *Curves and surfaces for CAGD: a practical guide*. Morgan Kaufmann, 2002.
[32] Gurobi Optimization, LLC, "Gurobi Optimizer Reference Manual," 2024.
[33] L. Schmid, O. Andersson, A. Sulser, P. Pfreundschuh, and R. Siegwart, "Dynablox: Real-time detection of diverse dynamic objects in complex environments," *IEEE Robotics and Automation Letters*, vol. 8, no. 10, pp. 6259–6266, 2023.
[34] S. Akhlaghi, N. Zhou, and Z. Huang, "Adaptive adjustment of noise covariance in kalman filter for dynamic state estimation," in *2017 IEEE Power & Energy Society General Meeting*, pp. 1–5, 2017.
[35] R. Firoozi, A. Mir, G. S. Camps, and M. Schwager, "Oa-mpc: Occlusion-aware mpc for guaranteed safe robot navigation with unseen dynamic obstacles," *IEEE Transactions on Control Systems Technology*, vol. 33, no. 3, pp. 940–951, 2025.
[36] A. Wu and J. P. How, "Guaranteed infinite horizon avoidance of unpredictable, dynamically constrained obstacles," *Autonomous robots*, vol. 32, no. 3, pp. 227–242, 2012.
[37] D. C. Liu and J. Nocedal, "On the limited memory bfgs method for large scale optimization," *Mathematical programming*, vol. 45, no. 1, pp. 503–528, 1989.
[38] K. Chen, R. Nemiroff, and B. T. Lopez, "Direct lidar-inertial odometry: Lightweight lio with continuous-time motion correction," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3983–3989, 2023.
[39] L. Meier, D. Honegger, and M. Pollefeys, "Px4: A node-based multithreaded open source robotics framework for deeply embedded platforms," in *2015 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6235–6240, 2015.
[40] L. Meier, P. Tanskanen, F. Fraundorfer, and M. Pollefeys, "Pixhawk: A system for autonomous flight using onboard computer vision," in *2011 IEEE International Conference on Robotics and Automation*, pp. 2992–2997, 2011.